



UNIVERSIDADE FEDERAL DO CEARÁ
CAMPUS QUIXADÁ
CURSO DE GRADUAÇÃO EM CIÊNCIA DA COMPUTAÇÃO

FRANCISCO FAGNER FERREIRA MESQUITA

**VANSI: UMA FERRAMENTA PARA O ENSINO DE ALGORITMOS DE ANÁLISE
SINTÁTICA**

QUIXADÁ

2026

FRANCISCO FAGNER FERREIRA MESQUITA

VANSI: UMA FERRAMENTA PARA O ENSINO DE ALGORITMOS DE ANÁLISE
SINTÁTICA

Relatório final apresentado à Universidade
Federal do Ceará, como requisito parcial para
obtenção do grau de Bacharelado em Ciência da
Computação.

Orientador: Prof. Dr. João Marcelo Uchôa de
Alencar

QUIXADÁ
18 de junho de 2026

RESUMO

A disciplina de compiladores é componente essencial dos cursos de Ciência da Computação, mas os algoritmos de análise sintática são notoriamente difíceis de ensinar e aprender devido à sua natureza abstrata e aos múltiplos passos intermediários. Este trabalho apresenta a VANSI (Visualização de Algoritmos de Análise Sintática), uma ferramenta educacional interativa e multiplataforma desenvolvida para auxiliar no ensino dos algoritmos de análise sintática LL(1), SLR e LR(1). Construída com o *framework Svelte* e empacotada para *desktop* e *mobile* por meio do *Tauri* e do *Capacitor*, a VANSI oferece execução passo a passo dos algoritmos com animações, autômatos interativos, árvores sintáticas, tabelas de parsing e pseudocódigo com *breakpoints*. A ferramenta foi avaliada por meio de um estudo de métodos mistos com 23 estudantes da Universidade Federal do Ceará, Campus Quixadá, combinando *logs* automatizados de interação com questionários estruturados. Os resultados indicam alta usabilidade (média 4,2/5) e eficácia pedagógica (média 4,4/5), com 91% dos participantes expressando satisfação e 90% manifestando intenção de reutilização. A VANSI preenche lacunas identificadas em ferramentas existentes ao oferecer maior interatividade e visualização detalhada dos processos internos dos algoritmos.

Palavras-chave: compiladores. algoritmos de análise sintática. visualização de algoritmos. tecnologia educacional. Svelte.

ABSTRACT

Compilers are a fundamental subject in Computer Science curricula, yet parsing algorithms are notoriously difficult to teach and learn due to their abstract nature and multiple intermediate steps. This work presents VANSI (Visualization of Syntax Analysis Algorithms), an interactive, cross-platform educational tool designed to support the teaching of LL(1), SLR, and LR(1) parsing algorithms. Developed using the Svelte framework and packaged for desktop and mobile via Tauri and Capacitor, VANSI provides step-by-step algorithm execution with animations, interactive automata, parse trees, parsing tables, and pseudocode with breakpoints. The tool was evaluated through a mixed-methods study involving 23 students from the Federal University of Ceará, Quixadá Campus, combining automated interaction logs with structured questionnaires. Results indicate high usability scores (mean 4.2/5) and strong pedagogical effectiveness (mean 4.4/5), with 91% of participants expressing satisfaction and 90% reporting intention to reuse the tool. VANSI addresses gaps identified in existing tools by offering greater interactivity and detailed step-by-step visualization of internal algorithmic processes.

Keywords: compiler construction. parsing algorithms. algorithm visualization. educational technology. Svelte.

SUMÁRIO

1	INTRODUÇÃO	10
1.1	Objetivos	10
1.2	Estrutura do Trabalho	11
2	FUNDAMENTAÇÃO TEÓRICA	12
2.1	Compiladores	12
2.1.1	Fases do compilador	12
2.1.1.1	Análise léxica	13
2.1.1.2	Análise sintática	13
2.1.1.3	Análise semântica	13
2.1.1.4	Otimização	13
2.1.1.5	Geração de código	14
2.2	Analisadores sintáticos descendentes	14
2.2.1	Descendentes recursivos	14
2.2.2	Analisadores sintáticos LL(1)	14
2.3	Analisadores sintáticos ascendentes	15
2.3.1	Analisadores sintáticos SLR	15
2.3.2	Analisadores sintáticos CLR	16
2.4	<i>Svelte</i>	17
3	TRABALHOS RELACIONADOS	19
3.1	<i>Teaching Compilers: Automatic Question Generation and Intelligent Assessment of Grammars' Parsing</i>	19
3.2	<i>PAVT: a tool to visualize and teach parsing algorithms</i>	19
3.3	<i>A Web-Based Educational System for Teaching Compilers</i>	20
3.4	<i>A Tool for Visualization of Parsers: JFLAP</i>	21
3.5	Considerações	22
4	METODOLOGIAS	23
4.1	Definição dos requisitos	23
4.2	Definição da arquitetura do projeto	24
4.3	Implementação da interface de usuário	26
4.4	Integração dos algoritmos à ferramenta	29
4.5	Implementação das animações dos algoritmos	31
4.6	Avaliação da ferramenta	31
4.6.1	Coleta de dados	32
4.6.2	Análise dos dados	34
4.6.3	Ferramentas e considerações técnicas	35

5	RESULTADOS	36
5.1	Caracterização dos dados coletados	36
5.2	Análise dos padrões de interação	37
5.2.1	Distribuição do tempo por aba	37
5.2.2	Padrões de navegação e interação	37
5.3	Avaliação de usabilidade	39
5.4	Eficácia pedagógica	39
5.5	Avaliação geral	40
5.5.1	Sugestões e comentários	40
6	CONCLUSÃO	42
6.1	Considerações finais	42
6.2	Limitações	42
6.3	Trabalhos futuros	42
	REFERÊNCIAS	43
	APÊNDICE A - ALGORITMO FIRST	45
	APÊNDICE B - ALGORITMO FOLLOW	46
	APÊNDICE C - ALGORITMO CONSTRUÇÃO DA TABELA LL(1)	47
	APÊNDICE D - ALGORITMO ANÁLISE LL(1)	48
	APÊNDICE E - ALGORITMO CONSTRUÇÃO DO AUTÔMATO LR(0)	49
	APÊNDICE F - ALGORITMO CLOSURE LR(0)	50
	APÊNDICE G - ALGORITMO CONSTRUÇÃO DA TABELA SLR	51
	APÊNDICE H - ALGORITMO ANÁLISE SLR	52
	APÊNDICE I - ALGORITMO CLOSURE LR(1)	53
	APÊNDICE J - ALGORITMO CONSTRUÇÃO DO AUTÔMATO LR(1)	54
	APÊNDICE K - ALGORITMO CONSTRUÇÃO DA TABELA LR(1)	55

LISTA DE FIGURAS

1	Fases de um compilador UNIX	12
2	Fluxo de processamento do <i>framework Svelte</i>	17
3	Interface da ferramenta PAVT	20
4	Imagem da ferramenta <i>JFLAP</i>	21
5	Fluxograma das etapas	23
6	Arquitetura da aplicação em nuvem	24
7	Aba de entrada da gramática	26
8	Aba de visualização do algoritmo SLR	27
9	Aba de visualização do algoritmo LR(1)	28
10	Aba de visualização da análise de uma <i>string</i>	29
11	Janela de diálogo para copiar resultados	29
12	Linha de tempo de uma sessão de uso da ferramenta	38
13	<i>Heatmap</i> de cliques na aba Tarefa	38

LISTA DE QUADROS

1	Comparativo de trabalhos relacionados	22
2	Possíveis valores do campo <i>type</i>	32
3	Esquema dos dados obtidos pelo processamento dos <i>logs</i>	33
4	Estrutura de seções do questionário de feedback	34

LISTA DE TABELAS

1	Dimensão da base de código da ferramenta VANSI	25
2	Perfil dos participantes da avaliação (N=23)	36
3	Volume de dados coletados durante a avaliação	37
4	Avaliação de usabilidade da ferramenta (escala 1-5)	39
5	Avaliação dos recursos pedagógicos (escala 1-5)	40
6	Temas identificados nos comentários abertos	40

LISTA DE ALGORITMOS

1	First	45
2	Follow	46
3	Construção da Tabela LL(1)	47
4	Análise LL(1)	48
5	Construção do Autômato LR(0)	49
6	Closure LR(0)	50
7	Construção da Tabela SLR	51
8	Análise SLR	52
9	Closure LR(1)	53
10	Construção do Autômato LR(1)	54
11	Construção da Tabela LR(1)	55

GLOSSÁRIO

AV *Algorithm Visualization.* 10

CLR *Canonical Left-to-right Rightmost derivation.* 10, 16, 19

CSS *Cascading Style Sheets.* 21

CSV *Comma-Separated Values.* 32

CYK *Cocke-Younger-Kasami.* 19

DOM *Document Object Model.* 17

HTML *HyperText Markup Language.* 21, 30

JFLAP *Java Formal Language Applet Platform.* 21, 22

JS *JavaScript.* 21

LALR *Look-Ahead Left-to-right Rightmost derivation.* 19

LL *Left-to-right Leftmost derivation.* 10, 14, 15, 19, 21, 22, 37, 42

LR *Left-to-right Rightmost derivation.* 15, 16, 22, 24, 37, 42, 51, 55

PAVT *Parser Algorithm Visualization Tool.* 19

SLR *Simple Left-to-right Rightmost derivation.* 10, 15, 16, 19, 21, 22, 37, 42

SSR *Server Side Rendering.* 24

SVG *Scalable Vector Graphics.* 30, 42

1 INTRODUÇÃO

A disciplina de compiladores está presente em muitas grades curriculares de cursos de ciência da computação pelo mundo. Dentro dessa disciplina são ensinados vários algoritmos de análise sintática. No entanto, aprender o funcionamento desses algoritmos é uma tarefa difícil para os alunos, assim como também é difícil para os professores ensinarem esse assunto (Sangal; Kataria; Tyagi *et al.*, 2018).

Tendo em mente a dificuldade no ensino sobre compiladores, foram criadas ferramentas de visualização de algoritmos(ou *Algorithm Visualization (AV)*). Essas ferramentas procuram ajudar no ensino de algoritmos que fazem parte das fases da compilação e, ao serem usadas com alunos, tiveram uma resposta positiva. Elas podem auxiliar na compreensão do conteúdo não só por meio da visualização do funcionamento dos algoritmos, mas também pelas respostas instantâneas, as quais podem ser usadas pelos estudantes para validar resultados que são difíceis de construir manualmente. (Jain; Goyal; Chakraborty, 2017).

O interesse pelo uso de AV's cresceu nos últimos anos. Com isso, também houve o surgimento de tecnologias de desenvolvimento *web* modernas e os avanços na capacidade e qualidade dos gráficos de *browsers*. Esses avanços permitiram um desenvolvimento *web* mais fácil e a criação de várias aplicações baseadas inteiramente na *web* (Romanowska *et al.*, 2018). Dentre os avanços no desenvolvimento *web* está a criação de *frameworks web* que reutilizam componentes e implementam funcionalidades para acelerar a criação de aplicações *web* (Uppal; Srivastava; Saini, 2022). Um exemplo de *framework web* é o *Svelte*¹ que foi utilizado para o desenvolvimento da ferramenta apresentada neste trabalho.

Embora esse tema tenha já sido abordado em trabalhos similares como o proposto por Sangal, Kataria e Tyagi (2017), ainda há avanços que podem ser alcançados como será discutido na Seção de trabalhos relacionados. Com isso, esse trabalho apresenta o desenvolvimento da ferramenta de visualização de analisadores sintáticos VANSI².

1.1 Objetivos

O objetivo deste trabalho é desenvolver uma ferramenta que auxilie na compreensão do funcionamento da análise sintática e quais os processos necessários para obter a saída de cada passo de seus algoritmos. Este trabalho tem os seguintes objetivos específicos, criar a ferramenta VANSI usando *Svelte*, desenvolver uma forma de visualização para os algoritmos *Canonical Left-to-right Rightmost derivation (CLR)*, *Simple Left-to-right Rightmost derivation (SLR)* e *Left-to-right Leftmost derivation (LL)* e validar a ferramenta fazendo uma avaliação com alunos.

¹<https://svelte.dev/>

²<https://vansi.netlify.app>

1.2 Estrutura do Trabalho

Este trabalho está organizado em seis capítulos, além das referências bibliográficas e dos apêndices. A estrutura está disposta da seguinte forma: o Capítulo 2 apresenta a fundamentação teórica, abordando conceitos de compiladores, analisadores sintáticos descendentes e ascendentes, além do *framework* Svelte; o Capítulo 3 discute os trabalhos relacionados, comparando a ferramenta VANSI com outras soluções existentes; o Capítulo 4 detalha as metodologias adotadas, descrevendo os requisitos, a arquitetura do projeto, a implementação da interface e a integração dos algoritmos; o Capítulo 5 expõe os resultados obtidos por meio da avaliação de usabilidade e eficácia pedagógica com os estudantes; e, por fim, o Capítulo 6 apresenta as considerações finais, as limitações encontradas e as propostas para trabalhos futuros. Adicionalmente, os Apêndices A a K detalham os algoritmos de análise sintática utilizados na construção da ferramenta.

2 FUNDAMENTAÇÃO TEÓRICA

Neste Capítulo, são apresentados os conceitos centrais que serviram como base e guia para a elaboração deste trabalho. A estrutura deste Capítulo está disposta da seguinte forma: a Seção 2.1 aborda o conhecimento básico sobre compiladores; a Seção 2.2 trata dos analisadores sintáticos descendentes; a Seção 2.3 discute os analisadores sintáticos ascendentes; e, por fim, a Seção 2.4 detalha o *framework* utilizado para a construção da ferramenta VANSI.

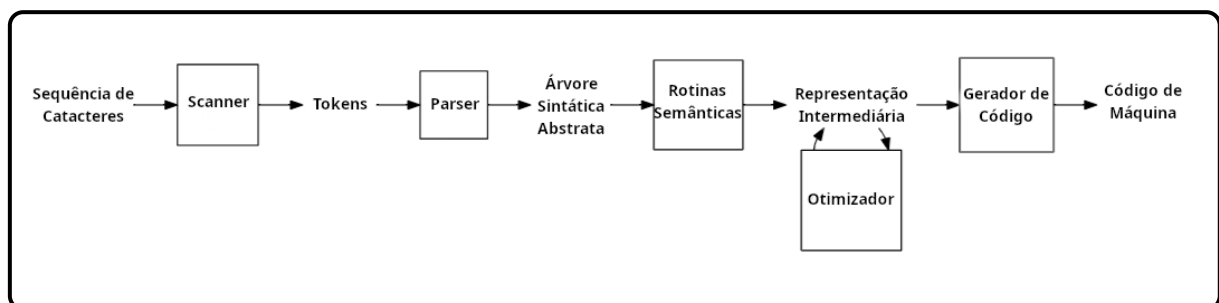
2.1 Compiladores

Programas que são executados em computadores são escritos no que é chamada de linguagem de máquina. Essa linguagem usa comandos simples que são interpretados pela máquina. Escrever em linguagem de máquina é uma tarefa passível de erro e cansativa. Por essa razão foram criados os compiladores. Os compiladores traduzem linguagens de alto nível para linguagem de máquina e indicam erros cometidos pelos programadores no código-fonte (Mogensen, 2024).

2.1.1 Fases do compilador

As fases de um compilador podem ser divididas de várias formas. Os trabalhos de Cooper e Torczon (2022), Mogensen (2024) e Thain (2020) dão suas definições das fases de um compilador, mas para este trabalho será seguida a definição de Thain (2020). As três primeiras fases do processo de compilação podem ser definidas como análise léxica, análise sintática e análise semântica. Elas são chamadas coletivamente de *front-end* do compilador (Mogensen, 2024). A fase de análise léxica gera *tokens* usados pela fase de análise sintática para validar que a entrada segue a estrutura da gramática da linguagem alvo e gerar uma árvore sintática para ser usada pelas próximas fases da compilação.

Figura 1: Fases de um compilador UNIX



Fonte: adaptado de Thain (2020).

2.1.1.1 Análise léxica

Na fase de análise léxica, o *scanner*, também chamado de *tokenizer*, consome texto simples de um programa e agrupa os caracteres individuais em sequências chamadas de *tokens*. Esse processo funciona de forma parecida com agrupar letras para formar palavras da linguagem natural. Esse agrupamento é feito usando expressões regulares implementadas através de autômatos.

2.1.1.2 Análise sintática

Análise de sintática será o foco do trabalho, sendo discutida de forma mais aprofundada nas próximas seções. A fase de análise sintática da compilação rearranja os *tokens* gerados pela fase de análise léxica, gerando assim uma estrutura chamada árvore sintática. As árvores sintáticas são estruturas de árvore como diz o nome. As folhas dessa árvore são os *tokens* e a leitura em ordem da árvore dá a sequência de *tokens* do texto de entrada dado ao analisador sintático. Ao construir a árvore sintática, o analisador sintático também checa se há erros de sintaxe no texto de entrada.

O programa que realiza a análise sintática é chamado analisador sintático ou *parser*. Os *parsers* podem ser classificados em dois tipos. O primeiro tipo é o *bottom-up* (ou ascendente) que funciona reduzindo os *tokens* às produções da gramática. O segundo tipo é o *top-down* (ou descendente) que segue o caminho oposto do *parser bottom-up* tentando encontrar produções correspondentes à estrutura da *string* de entrada comparando-a com as produções da gramática (Cooper; Torczon, 2022).

2.1.1.3 Análise semântica

Na fase de análise semântica, as rotinas semânticas percorrem a árvore sintática e buscam significado na entrada. Isso é feito a partir das regras da gramática e da relação entre os elementos da entrada. Depois das rotinas semânticas, a árvore de análise sintática é convertida em uma representação intermediária, a qual é uma versão simplificada de *assembly* que permite uma análise detalhada.

2.1.1.4 Otimização

Na fase de otimização, os otimizadores são aplicados na representação intermediária para tornar o programa mais rápido, menor e eficiente. Normalmente, os otimizadores recebem uma entrada em formato de representação intermediária e retornam um resultado no mesmo formato para que todos os otimizadores possam ser aplicados de forma independente e em qualquer ordem.

2.1.1.5 Geração de código

Na fase de geração de código, o gerador de código consome a representação intermediária otimizada e a transforma em um programa *assembly* concreto. Para otimizar o uso dos registradores físicos limitados e gerar instruções de montagem de maneira eficiente, o gerador de código precisa executar as tarefas de alocação de registradores, seleção de instruções e sequenciamento de instruções.

2.2 Analisadores sintáticos descendentes

Cooper e Torczon (2022), Mogensen (2024) e Thain (2020) falam sobre analisadores sintáticos descendentes, mas para este trabalho será seguida apenas a definição de Thain (2020). Analisadores sintáticos descendentes, também chamados *parsers top-down*, são métodos de análise sintática que iniciam a análise a partir do símbolo inicial da gramática. Eles fazem comparações entre os *tokens* do texto de entrada e os símbolos da gramática para encontrar a produção que deve ser escrita no lugar dos símbolos não-terminais. Essas comparações são feitas até sobrar apenas símbolos terminais. Esses símbolos terminais devem coincidir com a sequência de *tokens* da entrada, caso contrário, um erro é retornado.

2.2.1 Descendentes recursivos

O conjunto de gramáticas que podem ser analisadas usando algoritmos que utilizam apenas um não-terminal e o próximo símbolo da entrada é chamado conjunto de gramáticas LL(1). Uma das formas de fazer a análise dessas gramáticas é usando o analisador sintático descendente recursivo, o qual utiliza funções recursivas para cada não-terminal para processar a entrada. O analisador sintático descendente recursivo é uma técnica de implementação dos analisadores sintáticos LL(1) já tratados neste trabalho. Além disso, aproxima-se mais de um padrão de implementação para analisadores sintáticos LL(1) do que de um conceito algorítmico único com estruturas visualizáveis. Por essa razão, não está incluído na ferramenta VANSI.

2.2.2 Analisadores sintáticos LL(1)

Os analisadores sintáticos LL(1) são um tipo de analisador sintático descendente. Esses analisadores sintáticos levam em consideração um único *lookahead*. Para os analisadores sintáticos LL(1), o *lookahead* é o símbolo inicial do lado direito das produções. O *lookahead* é usado para decidir qual produção deve ser escrita. Por essa razão, apenas as gramáticas não ambíguas podem ser analisadas pelos analisadores sintáticos LL(1).

O conjunto dos símbolos iniciais das produções de uma gramática é chamado conjunto *first*. No algoritmo LL(1) o conjunto *lookahead* corresponde ao conjunto *first*. A construção do conjunto *first* pode ser feita seguindo o algoritmo no Apêndice A. As definições dos algoritmos foram tiradas do trabalho de Thain (2020).

O conjunto *follow* é o conjunto de símbolos terminais da gramática que podem ocorrer depois de qualquer uma das derivações de um não-terminal A . Esse conjunto também inclui o símbolo \$, usado para se referir ao fim de uma *string* de entrada w . O conjunto *follow* é usado no analisador LL para lidar com produções que derivam uma *string* vazia. A construção do conjunto *follow* pode ser feita seguindo o algoritmo no Apêndice B.

Uma tabela de análise sintática LL(1) pode ser usada para determinar as regras a serem usadas na análise de uma *string* para todas as combinações de não-terminais e *tokens* da *string*. A construção dessa tabela pode ser feita usando os conjuntos de *first* e *follow* seguindo o algoritmo no Apêndice C. Tendo a tabela de análise sintática LL(1) em mãos, é possível fazer a análise de uma sequência de *tokens* usando uma *stack*. O algoritmo no Apêndice D mostra a análise sintática usando a tabela LL(1).

2.3 Analisadores sintáticos ascendentes

Cooper e Torczon (2022), Mogensen (2024) e Thain (2020) falam sobre analisadores sintáticos ascendentes, mas para este trabalho será seguida a definição de Thain (2020). Os analisadores sintáticos ascendentes levam uma abordagem oposta aos analisadores sintáticos descendentes. Ao invés de começar com o símbolo inicial da gramática, os analisadores sintáticos ascendentes procuram sequências de *tokens* que façam par com o lado direito das produções da gramática e substituem as sequências de *tokens* pelo símbolo não-terminal do lado esquerdo da produção. Esse processo é repetido até que toda a sequência de *tokens* seja reescrita e apenas reste o símbolo inicial da gramática.

2.3.1 Analisadores sintáticos SLR

Há um conjunto de gramáticas que podem ser analisadas usando técnicas de *shift-reduce* sem uso de *lookahead*, esse conjunto de gramáticas pode ser chamado de conjunto *Left-to-right Rightmost derivation* (LR)(0). Usando um único *lookahead* junto as ações de *shift-reduce*, chegamos ao analisador sintático SLR que consegue analisar uma versão expandida do conjunto LR(0). As ações de *shift-reduce* são usadas para reduzir *tokens* de uma entrada a não-terminais. Quando uma sequência de *tokens* pode ser reduzida ao símbolo inicial da gramática, a análise da entrada teve sucesso, caso contrário, há um erro na entrada. e um único *lookahead*

Todas as ações de *shift-reduce* possíveis podem ser calculadas para uma gramática construindo um autômato LR(0), o qual também pode ser chamado de coleção de itens canônicos. Esse autômato guarda todas as possíveis posições de leitura das produções da gramática representadas por um ponto escrito no lado direito das produções.

As ações de *shift-reduce* são definidas pelas transições do autômato e pela posição de leitura. Caso uma transição seja feita com um terminal, a ação será de *shift*. Caso uma transição seja feita com um não-terminal, a ação será de *goto*. Caso a posição de leitura esteja no fim da produção, a ação será de *reduce*. O algoritmo no Apêndice E mostra como construir o autômato

LR(0) com o auxílio do *closure* mostrado no algoritmo no Apêndice F.

Quando um símbolo não-terminal produz um símbolo terminal, apenas um caminho para derivação é possível, já que um não-terminal não pode ser derivado. No entanto, quando um não-terminal produz outro não-terminal, o não-terminal produzido terá outras derivações. Por essa razão é preciso levar em consideração as produções dos não-terminais que estão sob a posição de leitura dentro da produção de outro não-terminal. *Closure* é o nome dado a ação de completar os estados do autômato adicionando essas produções.

Usando o autômato LR(0) podemos construir uma tabela de ações e *goto* para facilitar o acesso a essas informações durante o processo de análise sintática. Essa tabela pode ser calculada usando o algoritmo no Apêndice G. A análise sintática do analisador sintático SLR pode ser feita usando a tabela de ações e *goto* seguindo o algoritmo no Apêndice H.

2.3.2 Analisadores sintáticos CLR

O analisador sintático CLR (ou LR(1)) é um analisador sintático descendente para gramáticas LR(1). O analisador sintático CLR usa o autômato LR(1) para a construção da tabela de ações e *goto*. Esse autômato é parecido com o autômato LR(0), o que diferencia os dois é que todos os itens do autômato LR(1) têm uma anotação com o conjunto de *tokens* que podem aparecer depois desses itens. Esse conjunto de *tokens* é o conjunto *lookahead* para o analisador CLR.

A construção do autômato LR(1) segue o mesmo algoritmo da construção do autômato LR(0) com algumas modificações. A primeira produção a ser adicionada no primeiro estado do autômato vai ser adicionada com o *lookahead*. Esse *lookahead* tem \$ como único elemento. Ao computar o *closure* serão considerados dois casos:

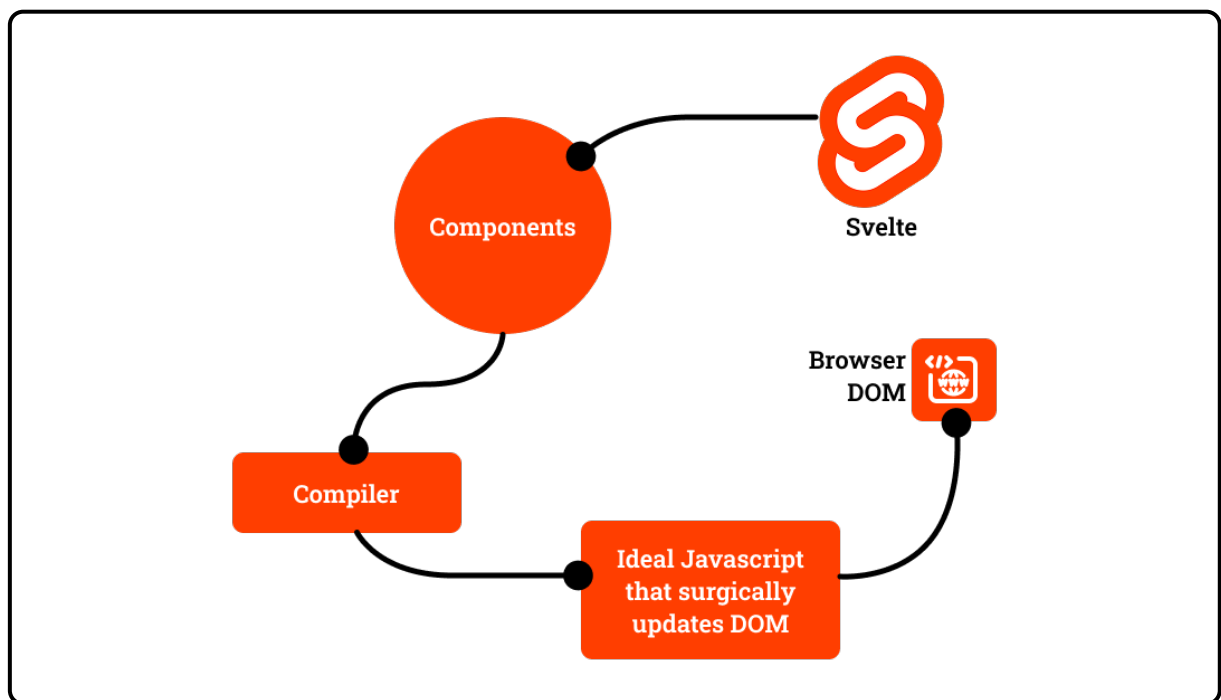
- Para produções da forma $A \rightarrow \alpha.B$ com *lookahead* de $\{L\}$, deverão ser adicionadas novas produções da forma $B \rightarrow \cdot\gamma$ com *lookahead* de $\{L\}$
- Para produções da forma $A \rightarrow \alpha.B\beta$, com *lookahead* de $\{L\}$, deverão ser adicionadas novas produções da forma $B \rightarrow \cdot\gamma$ com *lookahead* da seguinte forma:
 - Se β não produz ϵ , o *lookahead* é $First(G, \beta)$.
 - Se β produz ϵ , o *lookahead* é $First(G, \beta) \cup \{L\}$.

Com essas modificações chegamos aos algoritmos de *closure* LR(1) e construção do autômato LR(1) que podem ser vistos nos algoritmos nos Apêndices I e J respectivamente. O algoritmo de construção da tabela LR(1), mostrado no algoritmo no Apêndice K, também é parecido com o algoritmo de construção da tabela SLR. A única diferença é que o *lookahead* será usado no lugar do conjunto follow para definir as ações de *reduce*. Quanto ao algoritmo de *parsing* usando a tabela LR(1), ele é o mesmo algoritmo de *parsing* usando a tabela SLR.

2.4 Svelte

Svelte é um *framework* de componentes criado em 2016 por Harry Rich e pode ser considerado uma tecnologia recente em relação a outras similares (Krill, 2016). *Svelte* é semelhante aos *frameworks* *React* e *Vue*, mas tem uma abordagem bastante diferente no processamento de código. Os *frameworks* tradicionais usam código declarativo dirigido a estado (*declarative state-driven*). Isso aumenta a carga de processamento para o *browser* que precisa transformar essas estruturas declarativas em operações no *Document Object Model* (DOM) usando técnicas como *Virtual DOM* que é uma representação intermediária do DOM real (Harris, 2019a).

Figura 2: Fluxo de processamento do *framework* *Svelte*



Fonte: adaptado de Harris (2019a).

Ao contrário dos *frameworks* tradicionais, *Svelte* age como um compilador, funcionando em *build-time* para transformar os componentes criados em código *Javascript* imperativo altamente eficiente que atualiza o DOM apenas onde necessário. Isso permite escrever código para aplicações robustas sem necessidade de se preocupar muito com otimizações para que as aplicações sejam leves e performáticas. A Figura 2 mostra o fluxo de processamento do *framework* *Svelte*.

Além de ter código mais leve e performático, a quantidade de código escrito usando *Svelte* é menor em comparação a outros *frameworks*. Já que *Svelte* compila o código base, o *framework* é livre para escolher a forma como o código deve ser escrito e, para maior simplicidade o código em *Svelte* segue a sintaxe da linguagem *Javascript*. Tal coisa não é possível com outros *frameworks* como *React* que funciona em *runtime* e tem sua sintaxe limitada a isso, sendo necessário mais código para estar em conformidade com o funcionamento do *framework*. Um

exemplo disso é a atualização do estado de uma variável. Enquanto usando *Svelte* é apenas necessário usar o operador de atribuição para dar um novo valor à variável assim como na sintaxe *Javascript*. Usando *React* é necessário a utilização de funções chamadas *hooks* para a atribuição do novo valor (Harris, 2019b).

3 TRABALHOS RELACIONADOS

Neste Capítulo, estão descritas algumas ferramentas de visualização de algoritmos de análise sintática, suas funcionalidades e limitações.

3.1 *Teaching Compilers: Automatic Question Generation and Intelligent Assessment of Grammars' Parsing*

No trabalho de Muñoz *et al.* (2024) foi desenvolvida uma aplicação usada como *plug-in* no sistema de avaliação *SIETTE*. O objetivo da aplicação é gerar gramáticas livres de contexto, determinar se elas atendem aos requisitos LL(1) ou SLR, construir as tabelas de *parsing* e avaliar os alunos. Essa aplicação foi usada na Universidade de Málaga durante 7 anos para avaliar mais de mil alunos.

Para a criação de gramáticas livres de contexto foram usados blocos de construção. Esses blocos são conjuntos de produções que podem ter seus terminais substituídos por não-terminais representando outros blocos. Combinando diferentes blocos é possível serem obtidas gramáticas aleatórias. Para gerar as tabelas de *parsing* são implementados os algoritmos de construção dessas tabelas e algoritmos auxiliares. Apesar da equivalência entre gramáticas livres de contexto ser um problema indecidível, como as gramáticas livres de contexto usadas são pequenas, a aplicação consegue testar a equivalência entre elas usando um algoritmo de força bruta.

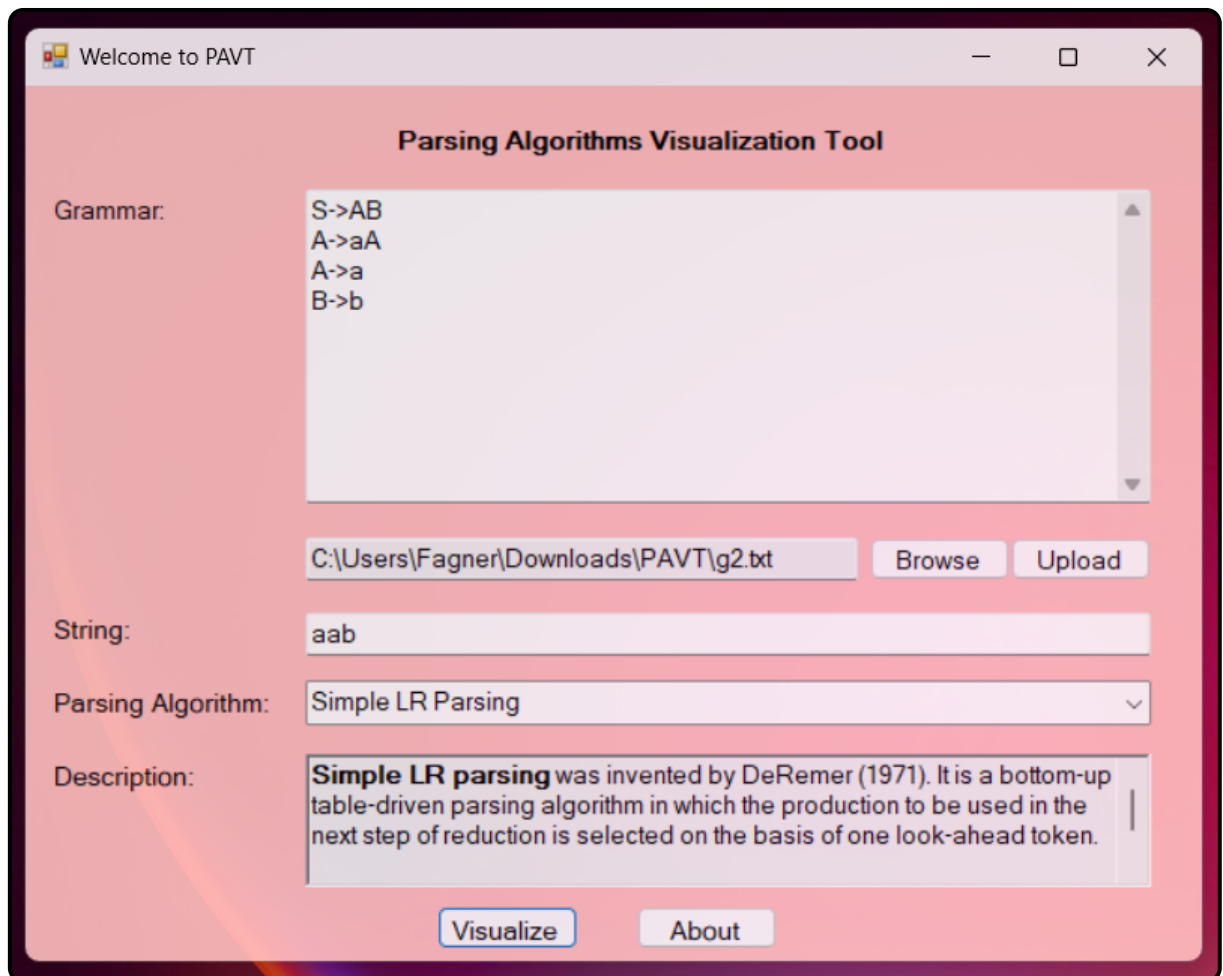
Após os dados da utilização da aplicação serem analisados, os autores concluíram que os testes gerados automaticamente têm dificuldade e resultados semelhantes aos obtidos com os testes criados por professores. Os autores também afirmam que os testes gerados automaticamente têm a vantagem de sempre diferir uns dos outros, requerendo um entendimento mais aprofundado dos alunos para resolução desses testes. Por fim, os autores citam como possibilidade de trabalhos futuros a criação de uma ferramenta com *feedback* gráfico e mais detalhado.

3.2 *PAVT: a tool to visualize and teach parsing algorithms*

No trabalho de Sangal, Kataria, Tyagi *et al.* (2018) foi introduzida a ferramenta *Parser Algorithm Visualization Tool* (PAVT) para ensinar seis algoritmos de análise sintática. Os algoritmos abordados na ferramenta são *predictive parsing*, *SLR parsing*, *CLR parsing*, *Look-Ahead Left-to-right Rightmost derivation (LALR) parsing*, *earley parsing* e *Cocke-Younger-Kasami (CYK) parsing*. PAVT mostra uma breve descrição dos algoritmos e dá o resultado dos passos do algoritmo em formato de texto. Para utilizar a ferramenta o usuário deve digitar uma *string* para ser analisada e a gramática alvo. Também é possível fazer *upload* de um arquivo de texto contendo a gramática. A interface da ferramenta pode ser vista na Figura 3.

PAVT tem módulos responsáveis pela visualização de cada algoritmo. Para todos é

Figura 3: Interface da ferramenta PAVT



Fonte: adaptada de Sangal, Kataria e Tyagi (2017).

feita a análise da *string* de entrada. Caso a *string* seja aceita, é construída a árvore sintática representada da esquerda para a direita. Além disso, todos os elementos presentes nos algoritmos são apresentados. Esses elementos são o conjunto *first*, conjunto *follow*, conjunto de itens canônicos, tabela de *parsing* e derivação mais à direita.

A ferramenta foi usada no curso de construção de compiladores e ao fim do curso o *feedback* dos alunos foi coletado. Os resultados obtidos indicaram que a ferramenta ajudou no aprendizado de algoritmos de análise sintática. Os autores afirmam que os resultados de cada passo dos algoritmos são dados em um formato comumente usado pelos professores e ajudam os estudantes a praticar e entender os algoritmos.

3.3 A Web-Based Educational System for Teaching Compilers

O trabalho de Stamenković e Jovanović (2024) foi feito na universidade de Pristina para criar uma versão *web* de um sistema de simulação chamado *ComVis*. Essa versão foi criada com módulos que ensinam as fases da compilação. O sistema já havia sido desenvolvido em

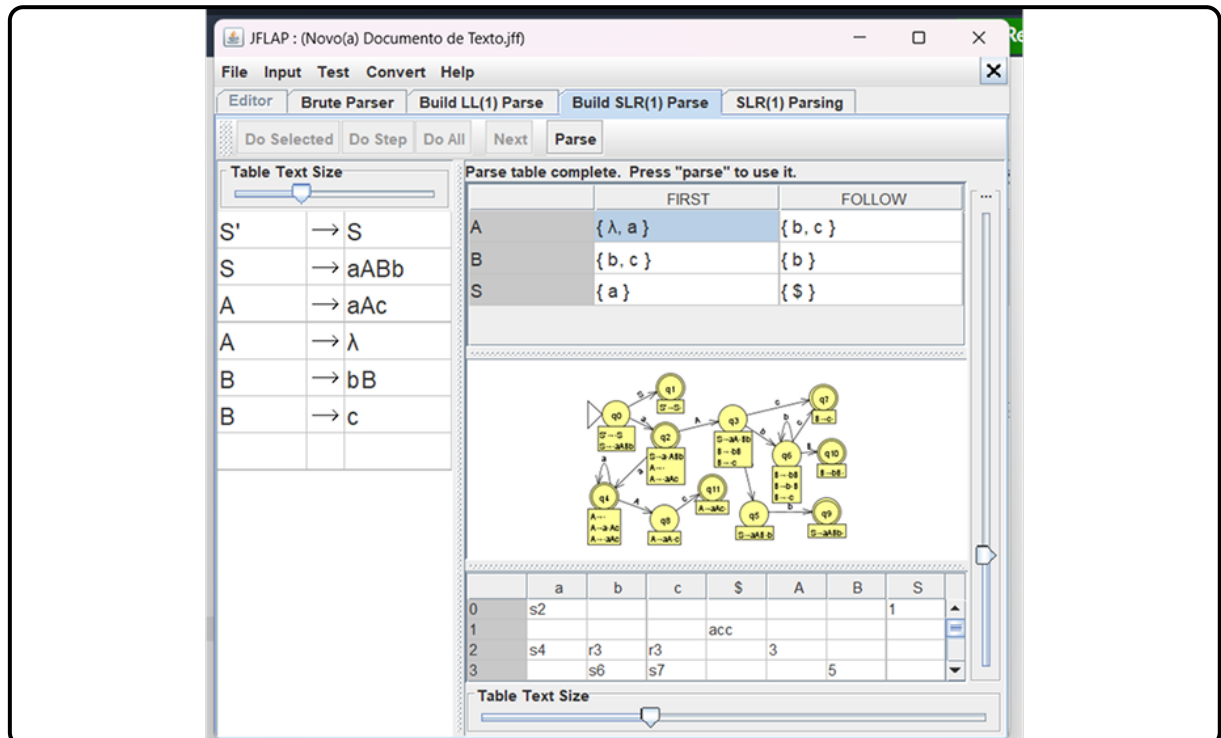
Java para *desktop*, no entanto, por questões de acessibilidade e melhor representação visual foi decidido criar a versão *web* do sistema. A motivação por trás desse trabalho foi a dificuldade dos alunos da universidade na disciplina de compiladores. Os autores também citam como a utilização de um *software* interativo pode ajudar na motivação.

O sistema foi desenvolvido usando *Java Server Page*. Já que a versão *desktop* do sistema foi feita em *Java*, grande parte da base de código pôde ser reutilizada. Outras tecnologias usadas foram *HyperText Markup Language* (HTML), *Cascading Style Sheets* (CSS), *JavaScript* (JS) e *Graphviz* para criação de gráficos e diagramas. O sistema aborda os *parsers* LL(1) e SLR, com foco apenas no último passo da análise sintática.

No estudo foi feita uma análise comparativa entre as versões *web* e *desktop* do sistema. A partir dessa análise os autores concluíram que a versão *web* criada tem melhor acessibilidade, visualização, controle da simulação e *feedback*. Também foi feita uma avaliação quantitativa da eficiência do sistema. Os resultados mostram que os estudantes que usaram o sistema tiveram melhor desempenho do que aqueles que não usaram o sistema.

3.4 A Tool for Visualization of Parsers: JFLAP

Figura 4: Imagem da ferramenta JFLAP



Fonte: Rodger e Duke University (2018).

Java Formal Language Applet Platform (JFLAP) é uma ferramenta *desktop* criada por Rodger e Duke University (2018) primariamente para a construção e simulação de autômatos e gramáticas livres de contexto. O trabalho de Devakumar, Naik e Sajja (2014) introduz os

algoritmos LL(1) e SLR na ferramenta. Dessa forma, JFLAP também pode ser usado para visualização dos algoritmos LL(1), SLR e de força bruta. Como mostra a Figura 4, a ferramenta apresenta os conjuntos *first* e *follow*, o autômato dos estados do analisador sintático e a tabela de ações. O processo de *parsing* também é disponibilizado assim como a árvore sintática.

3.5 Considerações

Apesar de já existirem ferramentas de visualização de *parsers*, algumas desvantagens ainda precisam ser consideradas. Uma limitação é que o conteúdo não tem muita interatividade. A ferramenta apresentada no trabalho de Sangal, Kataria, Tyagi *et al.* (2018), por exemplo, apresenta apenas um arquivo de texto. Isso pode dificultar a compreensão dos conceitos. Outra limitação é a falta de detalhamento do passo a passo dos algoritmos, por exemplo, a ferramenta apresentada no trabalho de Stamenković e Jovanović (2024) foca apenas no último passo da análise sintática, excluindo a parte de construção das tabelas de *parsing* e conjunto de itens. Isso impede que os estudantes acompanhem o funcionamento interno dos processos de análise. Essas lacunas representam oportunidades de melhoria para que as ferramentas de visualização de *parsers* se tornem ainda mais eficazes no apoio ao ensino e aprendizagem de análise sintática. O resumo do comparativo dos trabalhos citados anteriormente pode ser visto no Quadro 1.

Quadro 1: Comparativo de trabalhos relacionados

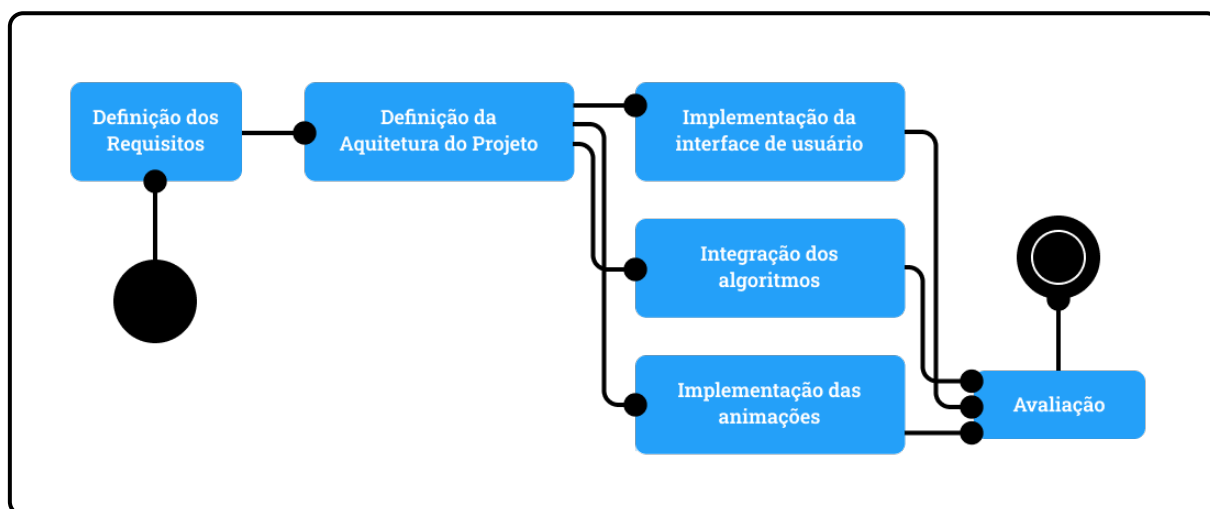
Trabalho	Algoritmos		
	LL(1)	SLR	LR(1)
Muñoz <i>et al.</i> (2024)	x	x	
Sangal, Kataria, Tyagi <i>et al.</i> (2018)		x	x
Stamenković e Jovanović (2024)	x	x	
Devakumar, Naik e Sajja (2014)	x	x	
Este trabalho	x	x	x

Fonte: fornecido pelo autor

4 METODOLOGIAS

Neste Capítulo serão apresentadas as metodologias usadas para o desenvolvimento deste trabalho. As etapas seguidas estão representadas no fluxograma da Figura 5.

Figura 5: Fluxograma das etapas



Fonte: fornecida pelo próprio autor

4.1 Definição dos requisitos

O primeiro passo no desenvolvimento de *software* é a definição dos requisitos. Com eles tem-se um direcionamento sobre o que deve ser feito. A partir da revisão bibliográfica foram definidos alguns requisitos básicos que deveriam estar presentes na ferramenta. Apesar das ferramentas compartilharem as mesmas funcionalidades básicas já definidas inicialmente, outras têm características interessantes que podem ser reaproveitadas. Como requisito não funcional foi definido oferecer suporte multi-plataforma, para *mobile*, *desktop* e *web*. Como requisitos funcionais foram definidos os seguintes:

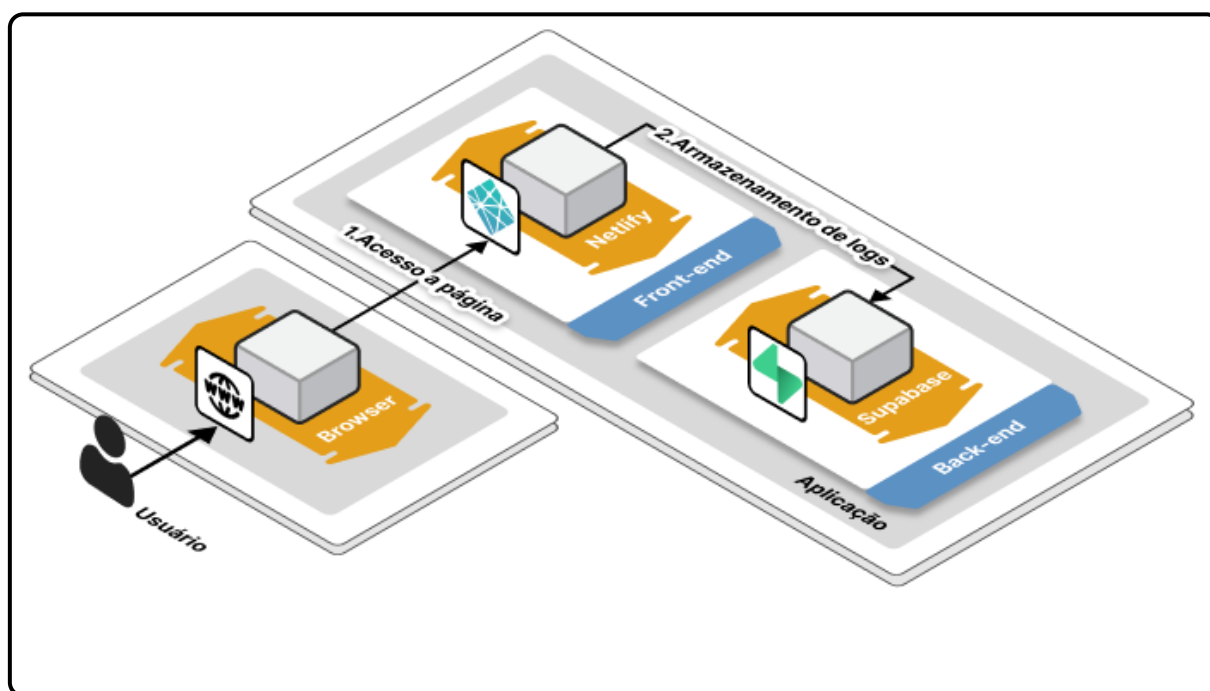
- RF1: Permitir que os usuários digitem uma gramática para ser analisada.
- RF2: Permitir que os usuários visualizem o estado das estruturas dos algoritmos.
- RF3: Permitir que os usuários avancem, retornem e reiniciem os passos da execução dos algoritmos.
- RF4: Permitir que os usuários selecionem o algoritmo a ser visualizado.
- RF5: Permitir que os usuários acessem o pseudocódigo dos algoritmos.
- RF6: Permitir que os usuários adicionem *breakpoints* no pseudocódigo.

- RF7: Permitir que os usuários visualizem o autômato dos algoritmos LR.
- RF8: Permitir que os usuários visualizem uma descrição dos botões da interface.
- RF9: Permitir que os usuários digitem uma *string* para ser analisada.
- RF10: Permitir que os usuários visualizem a análise de uma *string*.
- RF11: Permitir que os usuários visualizem a árvore sintática de uma *string*.
- RF12: Permitir que os usuários copiem em formato de texto o resultado dos algoritmos.

4.2 Definição da arquitetura do projeto

O projeto foi construído usando o *framework Svelte*, sem *Server Side Rendering (SSR)*, para ser possível o funcionamento *offline* da ferramenta. Já que a ferramenta não teria acesso ao servidor quando *offline*, não seria possível usá-lo para renderizar elementos. *Svelte* compila a base de código e cria uma coleção de arquivos estáticos que constituem a página *web* e a base para o suporte multi-plataforma da aplicação.

Figura 6: Arquitetura da aplicação em nuvem



Fonte: fornecida pelo próprio autor

O *framework Capacitor*¹ consome essa coleção de arquivos e cria um projeto para a plataforma *Android*. Esse projeto é usado para criar a versão *mobile* da ferramenta usando o *Android Studio*. O *framework Tauri*² constrói instaladores para *desktop* diretamente da coleção de

¹<https://capacitorjs.com>

²<https://tauri.app>

arquivos estáticos. A plataforma alvo dos instaladores é a plataforma em que eles são construídos. Já que o *framework* não tem suporte para construção *cross-platform* é necessária a utilização de máquinas virtuais para construir instaladores para diferentes plataformas *desktop*.

Para a versão *online* da aplicação, adotaram-se os serviços de computação em nuvem *Netlify*³ e *Supabase*⁴. O primeiro responsável por hospedar a aplicação e o segundo responsável pelo banco de dados. Ambos foram escolhidos pela facilidade de uso e por oferecerem planos gratuitos adequados a projetos de pequeno porte, como o proposto neste trabalho.

O código-fonte da aplicação foi armazenado em um repositório *Git*⁵ hospedado no *Github*⁶. Graças à integração entre *Netlify* e *Github*, a aplicação é atualizada automaticamente a cada *commit* realizado no repositório. Já o *Supabase* foi utilizado especificamente para armazenar os arquivos de *log* gerados durante a avaliação da ferramenta com os alunos.

Ao término do desenvolvimento, a base de código da ferramenta continha 127 arquivos-fonte, totalizando cerca de 16,5 mil linhas de código. A Tabela 1 apresenta a distribuição do esforço de implementação pelos principais módulos do sistema, evidenciando a separação entre a lógica dos algoritmos de análise sintática, os componentes de visualização e as estruturas gráficas interativas. O desenvolvimento do núcleo de funcionamento da ferramenta foi realizado em um período de 6 meses, com atualizações incrementais pelos 6 meses seguintes. O módulo que tomou mais tempo e esforço foi o módulo `lib/stepCalc`, módulo que engloba a integração dos algoritmos de parsing e a conciliação do fluxo das animações com o a execução dos algoritmos.

Tabela 1: Dimensão da base de código da ferramenta VANSI

Módulo	Arquivos	Linhas	Descrição
<code>lib/stepCalc</code>	10	4.346	Lógica dos algoritmos de <i>parsing</i>
<code>components/Algorithms</code>	12	1.780	Componentes para execução dos algoritmos
<code>components/Layout</code>	16	1.968	Elementos reutilizáveis de interface
<code>components/Tabs</code>	9	1.521	Sistema de abas
<code>components/Structures</code>	5	1.573	Renderização do autômato e árvore sintática
<code>components/Cards</code>	7	1.382	Caixas de estruturas de dados
<code>components/Info</code>	11	891	Diálogos informativos dos algoritmos
<code>routes</code>	5	301	Rotas da aplicação
Outros (Icons, lib, etc.)	52	2.713	Ícones, estilos globais e funções auxiliares
Total	127	16.475	Base de código completa da ferramenta

Fonte: fornecido pelo autor

A concentração de aproximadamente 26% do código em `lib/stepCalc` reflete a prioridade dada à corretude e ao detalhamento dos algoritmos de análise sintática, enquanto os 63% restantes distribuídos em `components` demonstram o esforço voltado à construção de

³<https://www.netlify.com>

⁴<https://supabase.com>

⁵<https://git-scm.com>

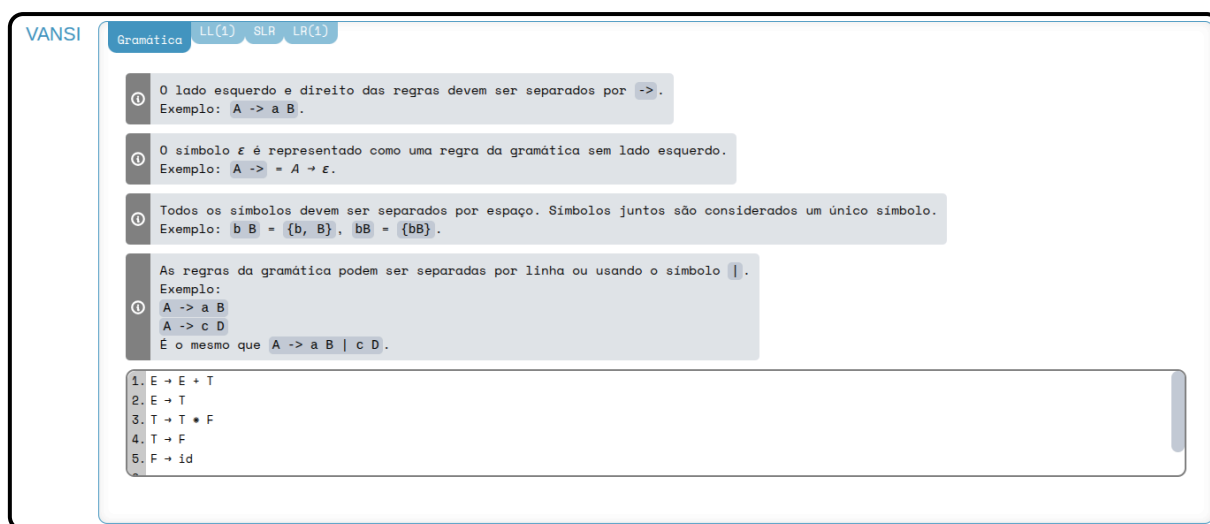
⁶<https://github.com/fagner02/parsers-svelte.git>

uma interface interativa e de múltiplas visualizações, conforme os requisitos definidos na Seção 4.1.

4.3 Implementação da interface de usuário

Nesta Seção estão descritos o *layout* da ferramenta e os elementos que o constituem. Começando pela barra de navegação, a qual contém abas que levam para as abas dos algoritmos e de entrada da gramática. A aba de entrada da gramática é a primeira. Nessa aba há apenas uma caixa de texto para digitar a entrada. Os elementos descritos neste parágrafo podem ser vistos na Figura 7.

Figura 7: Aba de entrada da gramática

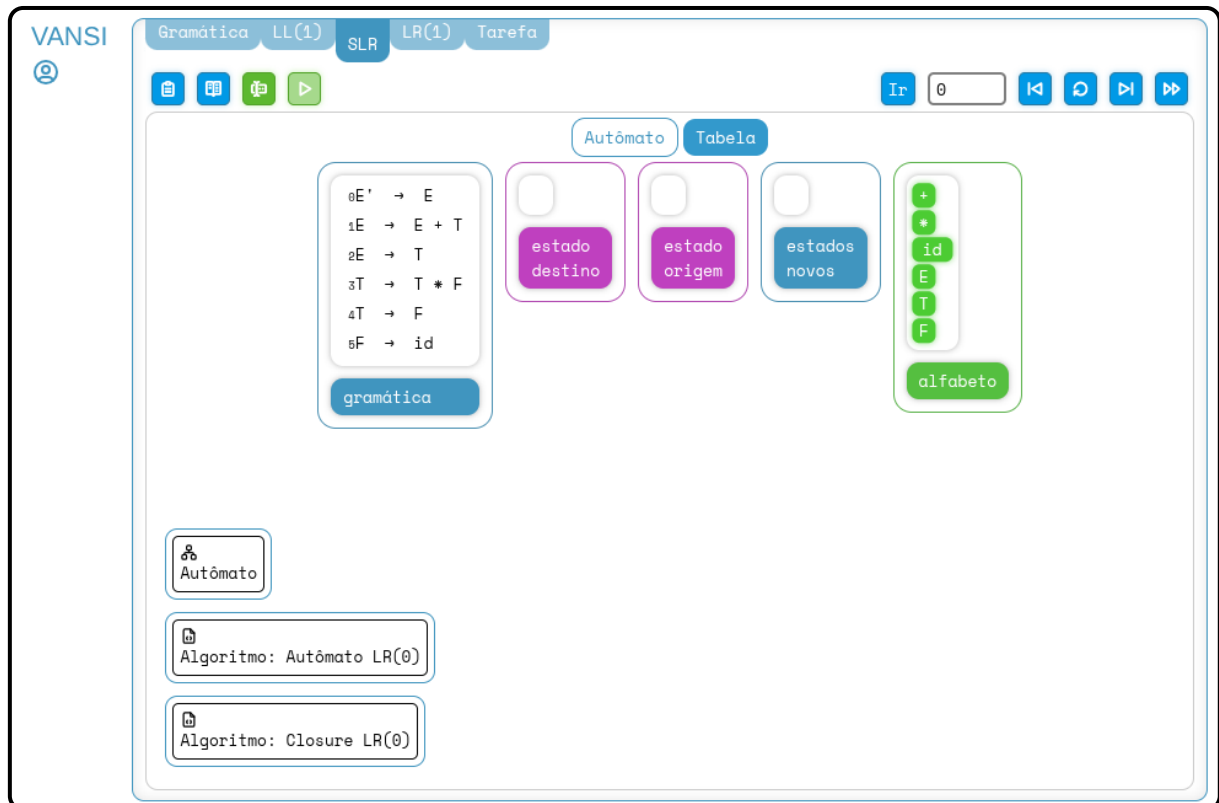


Fonte: fornecida pelo próprio autor

Na aba dos algoritmos há uma barra de ações com botões separados em dois grupos. No grupo da esquerda, os dois primeiros botões abrem respectivamente uma janela de diálogo com os resultados em formato de texto e uma janela de diálogo com uma descrição do algoritmo. Os outros dois botões do grupo da esquerda alternam entre a visualização da construção do *parser* e a visualização da análise de uma *string*. No grupo de botões da direita estão os controles de fluxo da execução do algoritmo. O primeiro botão vai para o passo da execução digitado no campo de texto ao lado desse botão. Os demais botões realizam as respectivas ações: ir para o passo anterior, reiniciar, ir para o passo seguinte e pular para o fim da execução.

Logo abaixo da barra de ações há uma lista de botões. Cada um deles abre a visualização de um algoritmo secundário usado na construção do parser selecionado e, abaixo desses botões estão os elementos que representam as estruturas de dados dos algoritmos. Os elementos descritos neste parágrafo podem ser vistos na Figura 8.

Figura 8: Aba de visualização do algoritmo SLR



Fonte: fornecida pelo próprio autor

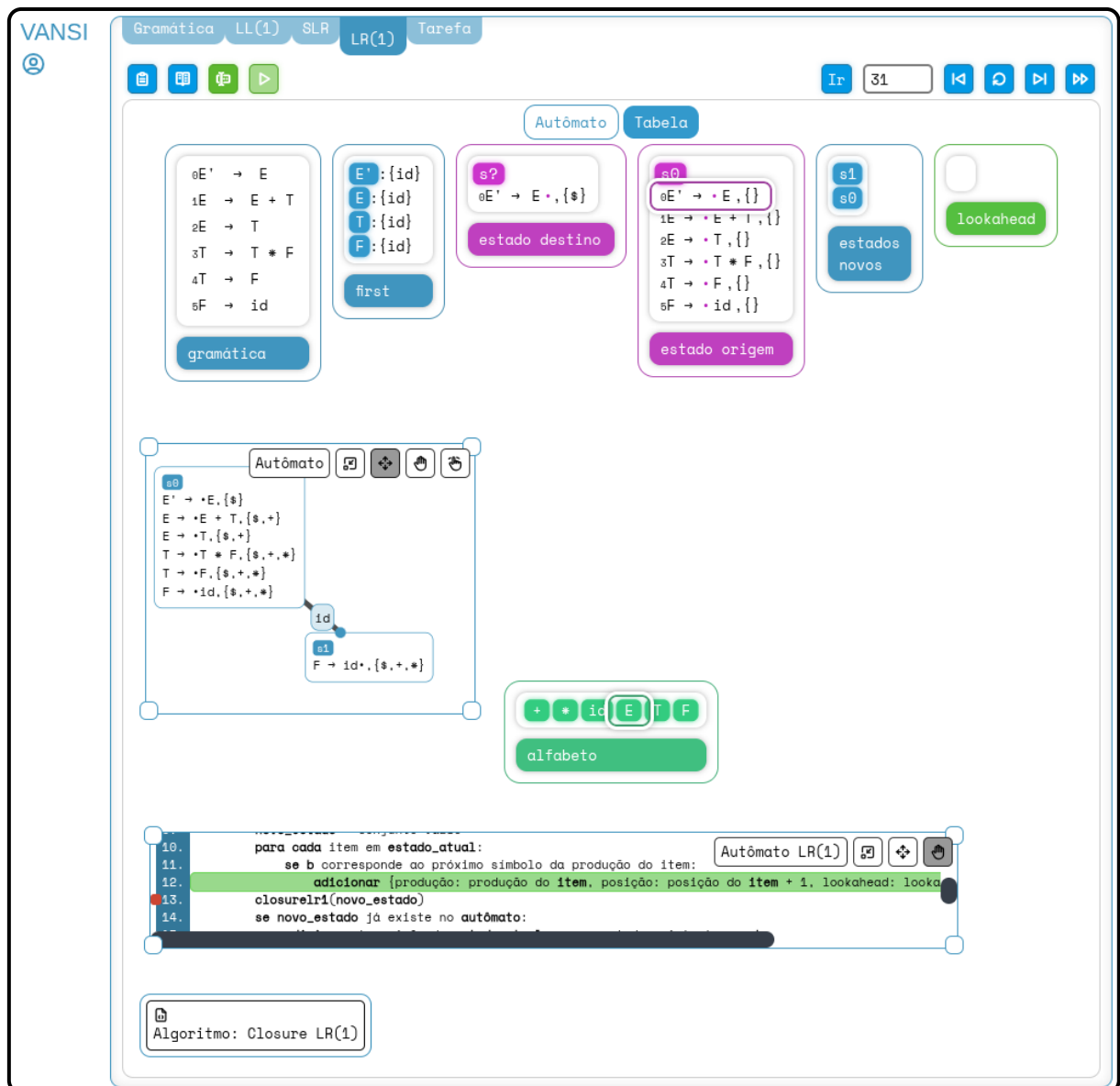
Ainda na aba dos algoritmos, existem janelas flutuantes onde são mostrados os pseudocódigos dos algoritmos e autômatos. Essas janelas flutuantes são abertas clicando em um botão flutuante com um ícone e título descrevendo a janela. Dentro dessas janelas, no canto superior direito, existe uma bandeja de ações com alguns botões. O primeiro botão minimiza a janela, o segundo ativa a ação de mover a janela ao clicar e arrastar ela e o terceiro ativa a interação com a janela. Os elementos descritos neste parágrafo e no parágrafo a seguir podem ser vistos na Figura 9.

Na janela flutuante que mostra um autômato, há um quarto botão na bandeja de ações. Esse botão permite que os estados do autômato sejam movidos. Na janela flutuante de pseudocódigo os números das linhas são clicáveis para que o usuário possa adicionar *breakpoints*. Por fim, nos cantos das janelas flutuantes existem pequenos quadrados que servem para redimensionar a janela.

Dentro da aba de análise de *strings*, no lado esquerdo, está uma área de visualização da árvore sintática e, no lado direito, está um campo de texto para digitar a *string* de entrada. Logo abaixo do campo de texto estão os elementos que representam as estruturas de dados dos algoritmos. Os elementos descritos neste parágrafo podem ser vistos na Figura 10.

As janelas de diálogo aparecem contidas na área de visualização da execução dos algoritmos. Acima de cada janela está um botão de fechar que cobre o comprimento inteiro da janela.

Figura 9: Aba de visualização do algoritmo LR(1)

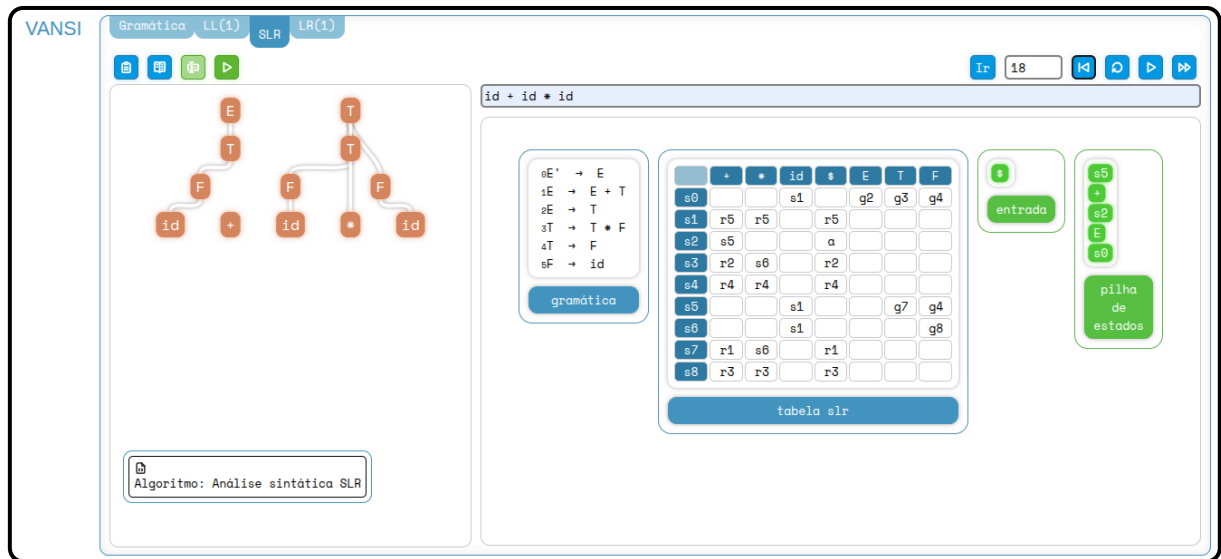


Fonte: fornecida pelo próprio autor

Dentro da janela de diálogo com os resultados dos algoritmos em formato de texto, estão os resultados de cada algoritmo divididos em seções e, ao lado do título de cada seção, está um botão de copiar. Na janela de diálogo de descrição do algoritmo tem apenas conteúdo textual. Os elementos descritos neste parágrafo podem ser vistos na Figura 11.

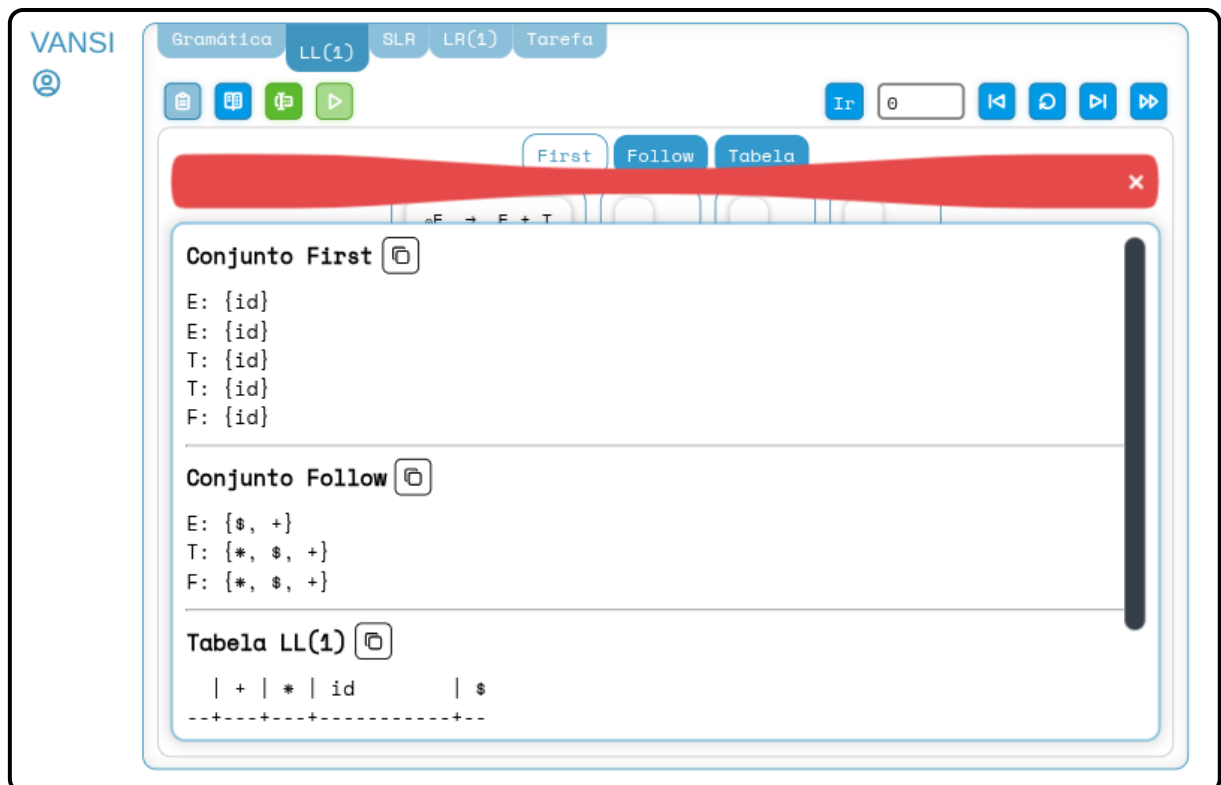
Por fim foi implementada um caixa de texto que mostra uma descrição dos elementos da interface de usuário ao passar o mouse por cima de um desses elementos. Essa caixa de texto funciona como um *tooltip*, sendo escondida quando o mouse sai de perto dos elementos.

Figura 10: Aba de visualização da análise de uma *string*



Fonte: fornecida pelo próprio autor

Figura 11: Janela de diálogo para copiar resultados



Fonte: fornecida pelo próprio autor

4.4 Integração dos algoritmos à ferramenta

O primeiro passo da integração dos algoritmos é a criação de elementos gráficos representando as estruturas de dados usadas nesses algoritmos. Foram criados, no total, seis elementos

para as estruturas de pilha, conjunto, tabela (ou matriz), autômato, estado de autômato e árvore sintática.

Os quatro primeiros elementos dos seis citados acima são caixas criadas com HTML, já que seus dados podem ser representados com simples listas de texto. Na parte inferior dessas caixas está o título do elemento e, na parte superior, estão elementos textuais, mostrando os dados das estruturas com destaques de cor nas chaves dos conjuntos e tabelas. Os dois últimos elementos são criados com *Scalable Vector Graphics* (SVG), já que seus dados precisam ser estruturados de forma complexa.

Com os elementos criados para representar as estruturas, é possível mostrar os passos da execução atualizando os dados desses elementos, conforme as mudanças feitas nas estruturas. Após a sincronização dos elementos com os passos de execução, é possível criar o controle de fluxo.

A primeira forma de implementação do controle de fluxo usada na aplicação foi usar uma única função que modifica as estruturas de dados e atualiza os elementos da interface em simultâneo. Essa função era controlada usando os mecanismos de execução assíncrona do *JavaScript* sendo eles *async*, *await* e *Promises*.

Nessa implementação, as pausas são feitas chamando uma *Promise* com retorno *undefined*, interrompendo a execução da função. A ação de ir para o próximo passo é feita rejeitando a *Promise*, o que retoma a execução da função. E por fim, as ações de voltar, pular e reiniciar são feitas retornando uma nova chamada da função. A nova chamada é executada e as pausas são puladas (as *Promises* são rejeitadas) até chegar no passo esperado.

O problema com a forma de implementação citada acima é que os passos da execução não são pré-calculados. Por essa razão, se a execução de um algoritmo tiver uma quantidade muito grande de passos, quando a ação de voltar ou pular for realizada, vários passos serão executados em sequência até chegar no passo esperado.

Dessa forma, dado o fato que as funções de atualização da interface gráfica também serão executadas, a aplicação vai requerer uma quantidade maior de poder computacional e vai parar de responder até que a execução dos passos seja finalizada. Esse problema não ocorria quando processando uma gramática pequena, mas, quando a gramática requeria muitos passos e operações das estruturas de dados, o problema se tornava certo de ocorrer.

Pela razão citada acima foi necessário criar uma segunda forma de implementação de controle de fluxo. Nessa implementação, o estado das estruturas em cada passo é salvo em uma *array* usando apenas as estruturas de dados sem modificar os elementos da interface gráfica. Dessa forma, a aplicação consegue pré-calcular os passos da execução sem muito esforço. Assim, quando as ações de controle de fluxo são executadas, os estados salvos são carregados e a interface gráfica é atualizada de acordo com eles.

4.5 Implementação das animações dos algoritmos

As mudanças de estados que ocorrem nos algoritmos podem ser melhor compreendidas se poderem ser visualizadas como transições ao invés de mostrar as mudanças saltando do estado inicial para o estado final. Usar animações torna a visualização das mudanças muito mais dinâmica.

As primeiras animações implementadas são as transições de estilo. Essas foram implementadas usando a propriedade *CSS transition* que aplica uma transição suave entre mudanças de estilo dos elementos como tamanho e cor. As animações seguintes foram implementadas usando *JavaScript* para serem atualizadas dinamicamente a medida que os elementos fossem modificados, mudando de tamanho ou posição na interface gráfica.

Começando com a animação de destaque dos dados dentro de um elemento da interface. Essa animação é usada quando os dados de um elemento são atualizados. Seja quando um item é adicionado a um elemento como inserções em *arrays* ou quando um item é usado como chave para acessar o item de outro elemento como um acesso a um elemento de um *map*. Essa animação põe um item da interface gráfica em destaque com um bloco colorido exibido atrás dele. Quando itens são adicionados às representações gráficas das tabelas de *parse*, as linhas e colunas também são destacadas com uma cor de fundo.

Várias estruturas de dados são representadas na interface gráfica por elementos com itens dispostos na vertical ou horizontal. Iterações com *for loops* feitas sobre esses itens também são representadas na interface gráfica. Para demonstrar cada iteração, uma moldura se movimenta sobre os elementos deixando claro a qual item cada iteração se refere.

Alguns dados são movidos entre elementos como no algoritmo *first* que move símbolos da direita de uma produção para o conjunto *first*. Para demonstrar essa transferência entre os elementos na interface gráfica, uma linha é exibida, estendendo-se do item de origem até o elemento de destino com uma animação de *morphing*. Essa animação é feita usando a biblioteca *anime.js*⁷ que calcula posições intermediárias entre os pontos de origem e destino de um *SVG Path* conforme a duração da animação.

4.6 Avaliação da ferramenta

A avaliação da ferramenta VANSI foi estruturada para investigar tanto sua usabilidade quanto sua eficácia pedagógica. O estudo adotou uma abordagem metodológica mista, combinando coleta automatizada de dados de interação com questionários estruturados de *feedback*. O estudo envolveu 23 estudantes voluntários que haviam cursado a disciplina de Compiladores do curso de Ciência da Computação do Campus Quixadá da Universidade Federal do Ceará.

⁷<https://animejs.com>

4.6.1 Coleta de dados

O primeiro passo da coleta de dados consistiu na criação do sistema de *log* integrado à própria ferramenta VANSI. Esse sistema registra automaticamente, diversos eventos de interação do usuário com a interface. Os *logs* são armazenados em formato *Comma-Separated Values* (CSV) com três campos base. O primeiro campo é o *timestamp* que registra o momento em que o evento ocorreu, o segundo campo é o *type* que indica o tipo de evento e o terceiro campo é o *desc* que fornece uma informação adicional do evento. Os possíveis valores do campo *type* estão descritos no Quadro 2

Quadro 2: Possíveis valores do campo *type*

Valor	Descrição
mouse pos	Movimento do <i>mouse</i> . Contém coordenadas do <i>mouse</i> e dimensões da <i>viewport</i>
w click	Clique do <i>mouse</i> . Contém coordenadas do <i>mouse</i> e dimensões da <i>viewport</i>
tab change	Alteração de aba. Contém o identificador da nova aba ativa
algorithm change	Alteração de algoritmo. Contém o identificador do novo algoritmo
control flow	Fluxo de controle. Contém identificador do botão de fluxo de execução
open parse	Abertura da aba de parse. Contém o nome do algoritmo de análise sintática usado para a análise
close parse	Ação de fechar a aba de parse. Contém o nome do algoritmo de análise sintática usado para a análise
open popup	Abertura de uma janela de diálogo. Contém o nome da janela de diálogo aberta
close popup	Ação de fechar uma janela de diálogo. Contém o nome da janela de diálogo
resize float	Redimensionamento de janela flutuante. Contém o identificador da janela redimensionada e suas dimensões
move float	Movimento de janela flutuante. Contém o identificador da janela movida e suas novas coordenadas
open float	Abertura de uma janela flutuante. Contém o nome da janela flutuante aberta e suas dimensões
close float	Ação de fechar uma janela flutuante. Contém o nome da janela flutuante fechada

Fonte: fornecido pelo autor

A partir dos dados base registrados nos *logs* foram calculadas novas propriedades usadas para a análise da interação com a ferramenta. Para isso foi criado um *script Python* que lê os arquivos CSV usando a biblioteca *pandas*. Esse *script* separa as linhas do CSV pelo valor do campo *type* e extrai os valores contidos no campo *desc*. Após esse processamento foram obtidas as propriedades descritas no Quadro 3.

O segundo passo da coleta de dados consistiu numa atividade prática aplicada com os alunos voluntários. A atividade era composta por sete tarefas, elaboradas para exercitar as principais funcionalidades da ferramenta e avaliar sua completude como recurso pedagógico. A sequência de tarefas estruturou-se da seguinte forma:

1. Geração de gramática a partir do conjunto *First*: Os discentes receberam um conjunto

Quadro 3: Esquema dos dados obtidos pelo processamento dos *logs*

Campo	Tipo	Descrição
timestamp	DateTime	Data e hora do evento com precisão de milissegundos
type	String	Categoria do evento (clique, posição, mudança de aba, etc.)
active_tab	String	Identificador da aba ativa no momento da interação
mouse_x	Integer	Coordenada x do cursor na interface gráfica
mouse_y	Integer	Coordenada y do cursor na interface gráfica
win_x	Integer	Dimensão x do <i>viewport</i> da interface gráfica
win_y	Integer	Dimensão y do <i>viewport</i> da interface gráfica
window_id	String	Identificador da janela flutuante envolvida na interação

Fonte: fornecido pelo autor

- First* e devem fornecer uma gramática que gere esse conjunto, correspondendo à questão 1 da atividade.
2. Geração de gramática a partir do conjunto *Follow*: A partir de um conjunto *Follow* fornecido, os participantes devem construir uma gramática que gere esse conjunto, correspondendo à questão 2.
 3. Geração de gramática a partir da tabela *SLR*: Dada uma tabela *SLR*, os discentes devem fornecer uma gramática que gere essa tabela, correspondendo à questão 3.
 4. Geração de gramática a partir da tabela *LR(1)*: A partir de uma tabela *LR(1)* fornecida, os participantes devem construir uma gramática que gere essa tabela, correspondendo à questão 4.
 5. Cálculo dos itens canônicos de uma gramática *LR(0)*: Dada uma gramática, os discentes devem fornecer o conjunto de itens canônicos *LR(0)* da gramática, correspondendo à questão 5.
 6. Análise sintática *LL(1)*: Os participantes devem mostrar os passos da análise sintática *LL(1)* da cadeia “a b a c” para uma gramática fornecida, correspondendo à questão 6.
 7. Construção de gramática *LR* para entradas específicas: Finalizando com a construção de uma gramática *LR* que aceita as entradas especificadas, correspondendo à questão 7.

Complementando os dados objetivos de interação, o terceiro passo de coleta consistiu em um questionário estruturado de *feedback* aplicado imediatamente após a conclusão da atividade prática. O questionário, composto por 21 questões, foi organizado em quatro seções principais, conforme detalhado no Quadro 4. A atividade combinou escalas *Likert* de cinco pontos para questões quantitativas e alternativas fechadas para questões categóricas, reservando uma questão aberta final para comentários e sugestões livres.

A seção de informações gerais buscou caracterizar o perfil dos participantes em relação à experiência prévia e nível de conhecimento. A seção de interface e usabilidade avaliou aspectos como facilidade de navegação, clareza dos elementos e organização do *layout*. A seção de recursos pedagógicos trata da utilidade percebida das diferentes visualizações, animações e mecanismos de controle. Por fim, a avaliação geral coletou impressões subjetivas sobre a ferramenta e intenções de uso futuro.

Quadro 4: Estrutura de seções do questionário de feedback

Seção	Nº Questões	Objetivo Específico
Informações Gerais	3	Caracterização do perfil e experiência prévia dos participantes
Interface e Usabilidade	7	Avaliação da navegabilidade, clareza visual e organização dos elementos
Recursos Pedagógicos	7	Análise da efetividade das visualizações, animações e controles didáticos
Avaliação Geral	4	Coleta de impressões holísticas e intenções de uso futuro

Fonte: fornecido pelo autor

4.6.2 Análise dos dados

A análise dos dados coletados adotou uma abordagem de métodos mistos, integrando técnicas quantitativas e qualitativas para uma compreensão multidimensional dos resultados. A análise quantitativa concentrou-se inicialmente na estatística descritiva das métricas de uso extraídas dos *logs*, calculando médias, medianas e desvio padrão para os diferentes tipos de interação.

Além disso, técnicas de visualização foram empregadas com a geração de *heatmaps* de distribuição de cliques e linhas do tempo de interação, facilitando a identificação de padrões comportamentais. A identificação desses padrões foi feita manualmente através da comparação entre as visualizações geradas e o *feedback* dos participantes.

Os *heatmaps* de cliques foram gerados utilizando as coordenadas de clique registradas nos *logs*, aplicando um mapa de calor sobre uma imagem interface da ferramenta para destacar as áreas de maior interação. As imagens da interface presentes nos *heatmaps* foram geradas automaticamente por um *script Common JavaScript* executado no *Node.js*⁸. O *script* utiliza a biblioteca *puppeteer* para lançar uma instância de um navegador, acessar o *url* da ferramenta, navegar até a tela desejada e fazer uma captura de tela tudo de forma automatizada. O *script* está disponível publicamente no repositório do projeto⁹.

Os rótulos de tempo (*timestamps*) dos cliques nas abas foram usados para determinar qual aba estava ativa durante cada evento registrado. Isso permitiu a geração de *screenshots*

⁸<https://nodejs.org>

⁹<https://github.com/fagner02/parsers-svelte.git>

específicas de cada aba da aplicação. Os rótulos de tempo também permitiram o cálculo de médias de tempo gasto em cada aba da aplicação.

As linhas do tempo de interação foram construídas plotando os eventos registrados ao longo do tempo, permitindo a visualização de sequências e durações de uso dos diferentes componentes da interface gráfica. Os gráficos de linha de tempo também marcam as transições entre as abas que são identificadas por cliques em elementos de navegação registrados nos *logs*.

Por fim, a análise qualitativa focou nos comentários livres do questionário. Essa análise buscou identificar temas recorrentes, sugestões de melhoria e percepções subjetivas que complementassem os dados quantitativos. A análise feita a partir de múltiplas fontes, integrando dados de *logs*, respostas escalares e comentários textuais, permitiu uma validação cruzada dos achados e uma compreensão mais rica da experiência dos usuários.

4.6.3 Ferramentas e considerações técnicas

Para o processamento e análise dos dados, foram desenvolvidos *scripts* em *Python* que utilizam as bibliotecas *pandas* e *numpy* para manipulação e análise estatística dos dados tabulares, *matplotlib* e *seaborn* para geração de visualizações e gráficos. Todo o código desenvolvido para a análise está disponível publicamente no repositório do projeto, garantindo a transparência e reprodutibilidade dos procedimentos analíticos.

A condução do estudo também considerou aspectos éticos da pesquisa envolvendo seres humanos. Foi obtido o consentimento informado de todos os participantes, garantindo o anonimato completo dos dados pessoais e a utilização exclusiva das informações para fins acadêmicos e de pesquisa.

Esta metodologia de avaliação, ao integrar dados objetivos de interação com percepções subjetivas dos usuários, permitirá não apenas quantificar a aceitação da ferramenta, mas também compreender os mecanismos pelos quais a visualização interativa pode facilitar o aprendizado de algoritmos complexos de análise sintática. Os resultados dessa análise serão apresentados e discutidos no capítulo seguinte.

5 RESULTADOS

Este Capítulo apresenta os resultados obtidos na avaliação da ferramenta VANSI. A análise integra dados quantitativos de interação coletados automaticamente pela ferramenta com respostas qualitativas dos questionários de *feedback*. A estrutura deste Capítulo está disposta de seguinte forma: a Seção 5.1 fala sobre a caracterização dos dados coletados; a Seção 5.2 fala sobre a análise dos padrões de interação; a Seção 5.3 fala sobre a avaliação de usabilidade; a Seção 5.4 fala sobre a eficácia pedagógica; e, por fim, a Seção 5.5 fala sobre a avaliação geral.

5.1 Caracterização dos dados coletados

Os 23 participantes da avaliação apresentaram o perfil descrito na Tabela 2. Nota-se que a maioria (60,9%) não possuía experiência prévia com algoritmos de análise sintática, configurando uma amostra representativa do público-alvo da ferramenta.

Tabela 2: Perfil dos participantes da avaliação (N=23)

Característica		Quantidade	Percentual
Experiência prévia com <i>parsing</i>	Sim	9	39,1%
	Não	14	60,9%
Nível de conhecimento autorrelatado	Iniciante	8	34,8%
	Intermediário	12	52,2%
	Avançado	3	13,0%
Uso prévio da ferramenta	Sim	0	0%
	Não	23	100%

Fonte: fornecido pelo autor

Durante as 7,7 horas totais de atividade, o sistema de *log* da VANSI registrou 88.301 eventos de interação. Com 23 estudantes participantes, a média de 3.839 eventos por estudante indica alto engajamento durante a sessão de avaliação. O tempo médio de uso foi de 20,1 minutos. Estudantes sem experiência prévia gastaram em média 28,5 minutos, enquanto aqueles com experiência utilizaram 15,8 minutos, sugerindo que a familiaridade com os conceitos reduziu o tempo necessário para execução das sete tarefas.

A Tabela 3 detalha o volume de dados coletados por categoria de evento. Os eventos de *mouse* representaram a maioria dos registros, seguidos pelas interações com controles de fluxo e trocas de aba.

Tabela 3: Volume de dados coletados durante a avaliação

Tipo de Evento	Total	Média por Estudante
Eventos de mouse (posição + clique)	87.421	3.801
Trocas de aba	412	17,9
Manipulações de janelas flutuantes	147	6,4
Controles de fluxo utilizados	321	14,0
Total de eventos	88.301	3.839

Fonte: fornecido pelo autor

5.2 Análise dos padrões de interação

Nesta Seção são examinados os comportamentos de navegação e interação dos participantes com a ferramenta, com base nos registros automáticos de eventos. Os resultados estão organizados em duas subseções, distribuição do tempo por aba e padrões de navegação e interação.

5.2.1 Distribuição do tempo por aba

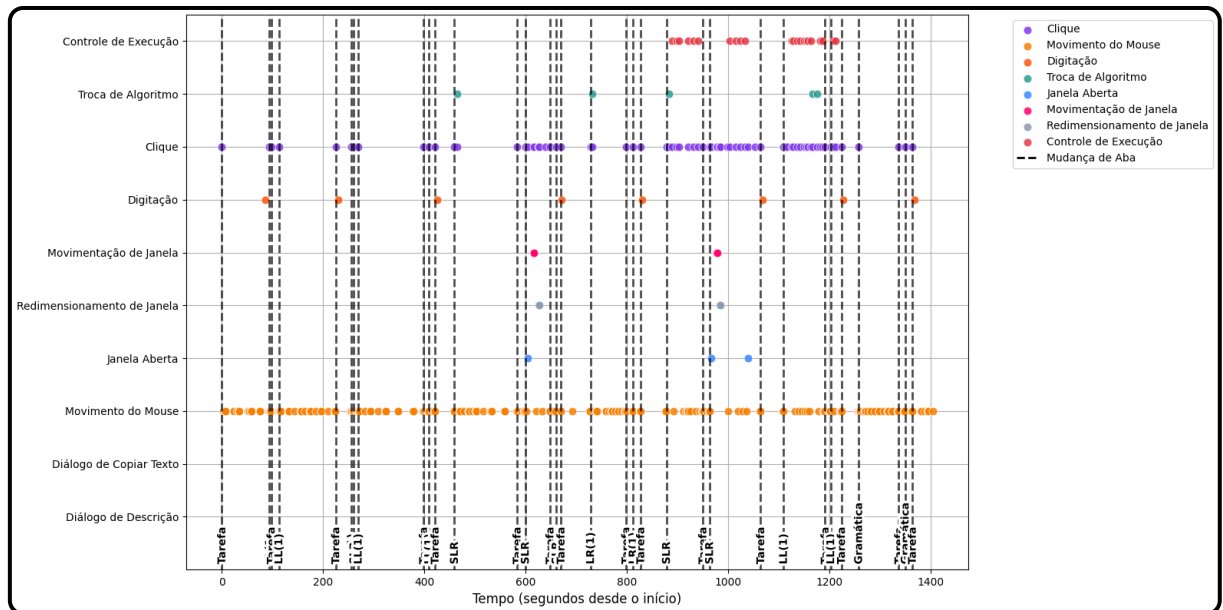
A distribuição do tempo gasto em cada aba da ferramenta revelou padrões consistentes entre os participantes. A aba SLR recebeu a maior atenção com 31,7% do tempo total, seguida pela aba LL(1) com 25,4% e LR(1) com 20,1%. A aba de Gramática, destinada à entrada inicial dos dados, consumiu 15,5% do tempo, enquanto a aba de Análise de *String*, utilizada principalmente para validação final, representou 7,3% do tempo total. Esta distribuição sugere que os algoritmos SLR e LL(1), tradicionalmente mais desafiadores para os estudantes, demandaram maior tempo de exploração e compreensão. A Figura 12 mostra a linha de tempo de um dos *logs*.

5.2.2 Padrões de navegação e interação

A análise dos *heatmaps* de cliques revelou padrões de interação distintos entre as abas. Na aba Tarefa, observou-se distribuição esparsa das interações pelos campos de entrada. Nas abas algorítmicas, porém, identificou-se concentração em elementos visuais específicos, a tabela de *parsing* na LL(1), o autômato LR(0) no SLR, e a árvore sintática na análise de *strings*.

A Figura 13 apresenta um *heatmap* de uma sessão de avaliação na aba SLR. Esse *heatmap* mostra a concentração de interações em elementos-chave da interface e padrão de uso esperado da ferramenta. Ainda que a maioria dos elementos tenha sido explorada, algumas funcionalidades como o botão “Ir” e a janela de pseudocódigo não receberam interações. Esse padrão de uso pode ser explicado pela quantidade grande de funcionalidades disponíveis e a falta de familiaridade com a ferramenta, sendo necessário um período de adaptação para explorar todas as funcionalidades. Também é possível que a atividade proposta não tenha demandado o uso de todas as funcionalidades, o que é um aspecto a ser considerado no *design* de tarefas para

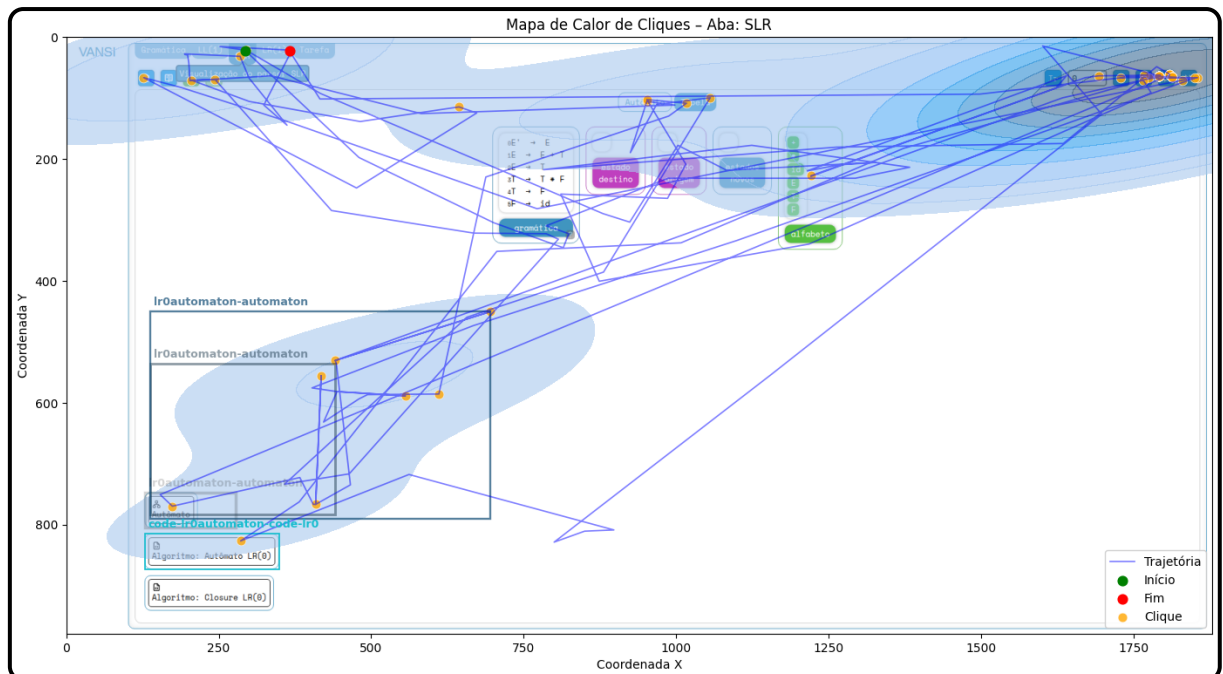
Figura 12: Linha de tempo de uma sessão de uso da ferramenta



Fonte: fornecido pelo próprio autor

futuras avaliações.

Figura 13: *Heatmap* de cliques na aba Tarefa



Fonte: fornecido pelo próprio autor

Os controles de fluxo foram utilizados extensivamente, com média de 14 ações por estudante. O botão “Avançar” representou 45,2% das ações, seguido por “Voltar” com 32,4% e “Pular para passo” com 12,1%. A predominância do uso sequencial (“Avançar” e “Voltar”) sobre o acesso direto (“Pular para passo”) sugere que os estudantes preferiram percorrer os algoritmos

de forma linear, revisando passos quando necessário, em vez de acessar etapas diretamente.

5.3 Avaliação de usabilidade

As respostas às questões de usabilidade apresentaram avaliação geral positiva. A Tabela 4 sumariza os resultados obtidos nas sete questões relacionadas à interface e usabilidade. A clareza dos elementos da interface obteve a maior nota média (4,3/5), com 91,0% das respostas nas categorias 4 ou 5. O diálogo para copiar resultados também recebeu avaliação elevada (4,5/5), indicando que a funcionalidade de exportação foi bem recebida. A facilidade de navegação obteve média de 4,1/5, com 85,0% de avaliações positivas.

Tabela 4: Avaliação de usabilidade da ferramenta (escala 1-5)

Aspecto Avaliado	Média	DP	% 4-5
Facilidade de navegação	4,1	±0,8	85%
Clareza dos elementos	4,3	±0,7	91%
Utilidade dos <i>tooltips</i>	4,0	±1,0	78%
Mover janelas flutuantes	4,4	±0,6	92%
Minimizar/abrir elementos	4,2	±0,9	83%
Organização das abas	4,1	±0,8	87%
Diálogo para copiar resultados	4,5	±0,5	96%
Média Geral	4,2	±0,7	87%

Fonte: fornecido pelo autor

A análise qualitativa dos *logs* permitiu identificar dois padrões comportamentais associados a dificuldades de usabilidade. Sete estudantes (30,4%) apresentaram um padrão de busca, caracterizado por múltiplas trocas rápidas de aba nos primeiros 5 minutos de uso, reduzindo progressivamente após período de adaptação. Nove estudantes (39,1%) apresentaram uso sub-ótimo de controles, utilizando predominantemente o botão “Avançar” sem explorar funcionalidades de pular para passo.

5.4 Eficácia pedagógica

Os recursos pedagógicos da VANSI receberam avaliação consistentemente alta. A Tabela 5 apresenta os resultados das sete questões relacionadas aos recursos pedagógicos da ferramenta. A visualização passo a passo obteve a maior nota média (4,7/5) e o maior consenso entre os participantes, com 100% das respostas concentradas nas categorias 4 e 5. Os controles de navegação (4,6/5) e a árvore sintática (4,5/5) também figuram entre os recursos mais bem avaliados, ambos com 96% e 91% de respostas positivas, respectivamente.

As animações e transições, implementadas para destacar mudanças nas estruturas de dados, receberam média de 4,4/5, com 91% das avaliações entre 4 e 5. O pseudocódigo com *breakpoints*, embora avaliado positivamente (4,2/5), obteve a menor média e o menor percentual

de aprovação entre os recursos pedagógicos (83%), possivelmente por demandar familiaridade prévia com a ferramenta. De modo geral, 91% das respostas do questionário indicaram satisfação com os recursos pedagógicos oferecidos.

Tabela 5: Avaliação dos recursos pedagógicos (escala 1-5)

Recurso Pedagógico	Média	DP	% 4-5
Visualização passo a passo	4,7	±0,5	100%
Animações e transições	4,4	±0,7	91%
Visualização do autômato	4,3	±0,8	83%
Árvore sintática	4,5	±0,6	91%
Controles de avançar/voltar	4,6	±0,5	96%
Pseudocódigo com <i>breakpoints</i>	4,2	±0,9	83%
Diálogos informativos	4,3	±0,7	87%
Média Geral	4,4	±0,7	91%

Fonte: fornecido pelo autor

5.5 Avaliação geral

A avaliação geral da ferramenta foi predominantemente positiva, com 86,9% dos participantes classificando-a como “Excelente” e 13,1% como “Boa”. Nenhum participante avaliou a ferramenta como “Ruim” ou “Péssima”. A consistência entre os diferentes tópicos avaliados sugere que a VANSI foi bem recebida tanto em aspectos técnicos quanto pedagógicos. As questões sobre intenção de uso futuro revelaram alta aceitação da ferramenta. 90,0% dos participantes responderam que usariam a ferramenta novamente para estudar algoritmos de compiladores, enquanto 85,0% afirmaram que a recomendariam para outros alunos.

5.5.1 Sugestões e comentários

A análise temática dos comentários abertos identificou quatro categorias principais de *feedback*, conforme apresentado na Tabela 6. Dois participantes elogiaram a interatividade da ferramenta, destacando que poder interagir com cada passo tornou a aprendizagem muito mais ativa e compreensível.

Tabela 6: Temas identificados nos comentários abertos

Tema	Frequência
Elogios à interatividade	2
Sugestões para experiência mobile	5
Sugestões de melhorias na interface	3

Fonte: fornecido pelo autor

Dentre as sugestões específicas, dois aspectos foram particularmente mencionados pelos participantes. Cinco usuários que acessaram a ferramenta por dispositivos móveis apontaram a necessidade de melhorias na experiência mobile, com comentários como “seria bom melhorar um pouco a visualização dos elementos pelo celular” e “no celular alguns botões ficam muito pequenos para clicar”.

Três estudantes sugeriram melhorias específicas na interface visual, com ênfase na clareza dos estados dos elementos interativos. Os comentários incluíam observações como “as cores dos botões ativos/inativos poderiam ser mais diferentes” e “fica confuso saber quando um botão está desabilitado”. Essas observações apontam para oportunidades de aprimoramento do relacionamento entre funcionalidade e aparência dos controles da interface, isto é, para melhorar a percepção de funcionalidade dos elementos da interface.

6 CONCLUSÃO

Este trabalho apresentou o desenvolvimento e a avaliação da VANSI, uma ferramenta interativa projetada para auxiliar o processo de ensino de algoritmos de análise sintática. A motivação para este projeto surgiu da complexidade inerente à disciplina de Compiladores e da carência de ferramentas que ofereçam uma visualização clara e passo a passo dos algoritmos de análise sintática.

6.1 Considerações finais

Os objetivos propostos inicialmente foram alcançados. A ferramenta foi desenvolvida utilizando tecnologias modernas de desenvolvimento *web* e suporta os algoritmos LL(1), SLR e LR(1). A avaliação da ferramenta foi realizada com um grupo de alunos da Universidade Federal do Ceará.

A análise quantitativa dos resultados demonstrou uma alta taxa de satisfação e intenção de uso futuro. As avaliações positivas entre a avaliação geral e a intenção de recomendação indica que a VANSI possui um valor percebido significativo pelos usuários. Além disso, a análise dos padrões de interação revelou que os alunos exploraram as diferentes funcionalidades de visualização.

6.2 Limitações

Apesar dos objetivos propostos terem sido alcançados, é importante reconhecer as limitações deste trabalho. A amostra de participantes foi relativamente pequena e composta por alunos voluntários. Além disso, a avaliação foi realizada sem grupo de controle, o que pode limitar a generalização dos resultados. A avaliação focou na satisfação e na intenção de uso, sem uma análise aprofundada do impacto da ferramenta no desempenho acadêmico dos alunos. Por fim, a avaliação não considerou a opinião de docentes sobre a ferramenta, o que poderia fornecer *insights* para melhorias futuras.

6.3 Trabalhos futuros

Considerando os resultados obtidos e as limitações deste trabalho, ainda há oportunidades de expansão para este estudo. Como trabalhos futuros, sugere-se a realização de uma avaliação mais profunda com grupos de controle, a inclusão de suporte para analisadores mais complexos, a implementação de uma *engine* de animação utilizando SVG para melhorar a performance das animações e, por fim, a otimização da interface para uso em *smartphones*.

REFERÊNCIAS

COOPER, K.D.; TORCZON, L. **Engineering a Compiler**. [S. l.]: Elsevier Science, 2022

DEVAKUMAR, S; NAIK, Ds; SAJJA, V Ramakrishna. A TOOL FOR VISUALISATION OF PARSERS: JFLAP. **Journal of Innovative Research and Solutions(JIRAS)**, v. 1, p. 1–4, mar. 2014

HARRIS, Rich. **Svelte 3: Rethinking reactivity**. 22 abr. 2019. <https://svelte.dev/blog/svelte-3-rethinking-reactivity>. Acesso em: 27 mai. 2024

HARRIS, Rich. **Write less code**. 20 abr. 2019. <https://svelte.dev/blog/write-less-code>. Acesso em: 27 mai. 2024

JAIN, Aashi; GOYAL, Archita; CHAKRABORTY, Pinaki. PPVT: a tool to visualize predictive parsing. **ACM Inroads**, Association for Computing Machinery, New York, NY, USA, v. 8, n. 1, p. 43–47, fev. 2017. ISSN 2153-2184

KRILL, Paul. **Slim, speedy Svelte framework puts JavaScript on a diet | InfoWorld**. Slim, speedy Svelte framework puts JavaScript on a diet. 2 dez. 2016. <https://www.infoworld.com/article/3146966/slim-speedy-svelte-framework-puts-javascript-on-a-diet.html>. Acesso em: 27 mai. 2024

MOGENSEN, Torben Ægidius. **Introduction to compiler design**. [S. l.]: Springer Nature, 2024

MUÑOZ, Ricardo Conejo *et al.* Teaching Compilers: Automatic Question Generation and Intelligent Assessment of Grammars' Parsing. **IEEE Transactions on Learning Technologies**, v. 17, p. 1734–1744, 2024

RODGER, Susan H; DUKE UNIVERSITY. **JFLAP**. 27 jul. 2018. www.jflap.org. Acesso em: 2 mai. 2024

ROMANOWSKA, Katarzyna *et al.* Towards Developing an Effective Algorithm Visualization Tool for Online Learning. In: 2018 IEEE SmartWorld, Ubiquitous Intelligence & Computing, Advanced & Trusted Computing, Scalable Computing & Communications, Cloud & Big Data Computing, Internet of People and Smart City Innovation (SmartWorld/SCALCOM/UIC/ATC/CBDCom/IOP/SCI). [S. l.: s. l.], 2018. P. 2011–2016. DOI: 10.1109/SmartWorld.2018.00336

SANGAL, Somya; KATARIA, Shreya; TYAGI, Twishi. **PAVT**. 9 fev. 2017. <https://sourceforge.net/projects/pavt/>. Acesso em: 2 mai. 2024

SANGAL, Somya; KATARIA, Shreya; TYAGI, Twishi *et al.* PAVT: a tool to visualize and teach parsing algorithms. **Education and Information Technologies**, Springer, v. 23, p. 2737–2764, 2018

STAMENKOVIĆ, Srećko; JOVANOVIĆ, Nenad. A Web-Based Educational System for Teaching Compilers. **IEEE Transactions on Learning Technologies**, v. 17, p. 143–156, 2024

THAIN, D. **Introduction to Compilers and Language Design: Second Edition**. [S. l.]: Amazon Digital Services LLC - Kdp, 2020

UPPAL, Tanya; SRIVASTAVA, Saumitya; SAINI, Kavita. Web Development Framework : Future Trends. **2022 4th International Conference on Advances in Computing, Communication Control and Networking (ICAC3N)**, p. 2181–2184, 2022. <https://api.semanticscholar.org/CorpusID:257809936>

APÊNDICE A - ALGORITMO FIRST

Algoritmo 1: First

Entrada: Gramática G , Símbolo X

Saída: Conjunto $first$

inicio

$first \leftarrow \emptyset$

se X é terminal **então**

$first \leftarrow \{X\}$

fim

senão

repetir

para cada regra $X \rightarrow Y_1 Y_2 \dots Y_k$ em G **fazer**

se $a \in \text{FIRST}(Y_1) \vee (a \in \text{FIRST}(Y_n) \wedge Y_1 \dots Y_{n-1} \Rightarrow^* \epsilon)$ **então**

$first \leftarrow first \cup \{a\}$

fim

se $Y_1 \dots Y_k \Rightarrow^* \epsilon$ **então**

$first \leftarrow first \cup \{\epsilon\}$

fim

fim

até que não haja mais mudanças

fim

retornar $first$

fim

APÊNDICE B - ALGORITMO FOLLOW

Algoritmo 2: Follow

Entrada: Gramática G , Símbolo A

Saida: Conjunto $follow$

inicio

$follow \leftarrow \emptyset$

se A é o símbolo inicial S **então**

$follow \leftarrow \{\$ \}$

fim

repetir

para cada produção $B \rightarrow \alpha A \beta$ em G **fazer**

se $\beta \neq \epsilon$ **então**

$follow \leftarrow follow \cup (\text{FIRST}(\beta) - \{\epsilon\})$

fim

se $\beta = \epsilon \vee \epsilon \in \text{FIRST}(\beta)$ **então**

$follow \leftarrow follow \cup \text{FOLLOW}(B)$

fim

fim

até que não haja mais mudanças

retornar $follow$

fim

APÊNDICE C - ALGORITMO CONSTRUÇÃO DA TABELA LL(1)

Algoritmo 3: Construção da Tabela LL(1)

Entrada: Gramática G

Saida: Tabela M

inicio

$M \leftarrow$ Tabela(Não-terminal, Terminal, Produção)

para cada regra $A \rightarrow \alpha$ em G **fazer**

para cada terminal $a \in \text{FIRST}(\alpha) - \{\epsilon\}$ **fazer**

$M[A, a] \leftarrow (A \rightarrow \alpha)$

fim

se $\epsilon \in \text{FIRST}(\alpha)$ **então**

para cada terminal $b \in \text{FOLLOW}(A)$ **fazer**

$M[A, b] \leftarrow (A \rightarrow \alpha)$

fim

fim

fim

retornar M

fim

APÊNDICE D - ALGORITMO ANÁLISE LL(1)

Algoritmo 4: Análise LL(1)

Entrada: Gramática G , Tabela M , String de entrada w

Saida: Sucesso ou erro

inicio

$pilha \leftarrow [\$, S]$

$token \leftarrow$ próximo token de w

enquanto $pilha \neq [\$]$ **fazer**

$X \leftarrow \text{topo}(pilha)$

se X é terminal **então**

se $X = token$ **então**

$\text{desempilhe}(pilha)$

$token \leftarrow$ próximo token de w

senão

retornar erro

fim

senão

se $M[X, token] = (X \rightarrow \alpha)$ **então**

$\text{desempilhe}(pilha)$

$\text{empilhe}(\alpha)$ da direita para a esquerda

senão

retornar erro

fim

fim

fim

se $token = \$$ **então**

retornar sucesso

senão

retornar erro

fim

fim

APÊNDICE E - ALGORITMO CONSTRUÇÃO DO AUTÔMATO LR(0)

Algoritmo 5: Construção do Autômato LR(0)

Entrada: Gramática G
Saida: Autômato M
inicio

```

   $M \leftarrow \emptyset$ 
   $s_0 \leftarrow \{[S' \rightarrow \cdot S]\}$ , Onde  $S$  é o símbolo inicial
   $s_0 \leftarrow \text{CLOSURE}(s_0)$ 
   $M \leftarrow M \cup \{s_0\}$ 
   $newStates \leftarrow \{s_0\}$ 
  enquanto  $newStates \neq \emptyset$  fazer
    selecione  $s \in newStates$ 
     $newStates \leftarrow newStates - \{s\}$ 
    para cada  $X \in T \cup NT$  fazer
       $s' \leftarrow \emptyset$ 
      para cada item  $[A \rightarrow \alpha \cdot X\beta] \in s$  fazer
         $s' \leftarrow s' \cup \{[A \rightarrow \alpha X \cdot \beta]\}$ 
      fim
      se  $s' = \emptyset$  então
        | continua para a próxima iteração
      fim
       $s' \leftarrow \text{CLOSURE}(s')$ 
      se  $s' \notin M$  então
        |  $M \leftarrow M \cup \{s'\}$ 
        |  $newStates \leftarrow newStates \cup \{s'\}$ 
      fim
      adicione transição  $s \xrightarrow{X} s'$ 
    fim
  fim
  retornar  $M$ 
fim
```

APÊNDICE F - ALGORITMO CLOSURE LR(0)

Algoritmo 6: Closure LR(0)

Entrada: Gramática G , Estado S **Saida:** Closure do estado S **inicio** $oldSize \leftarrow 0$ **repetir** $oldSize \leftarrow |S|$ **para cada** item $[A \rightarrow \alpha \cdot X\beta] \in S$ **fazer****para cada** produção $X \rightarrow \gamma \in G$ **fazer****se** $[X \rightarrow \cdot\gamma] \notin S$ **então** $S \leftarrow S \cup \{[X \rightarrow \cdot\gamma]\}$ **fim****fim****até que** $|S| = oldSize$ **retornar** S **fim**

APÊNDICE G - ALGORITMO CONSTRUÇÃO DA TABELA SLR

Algoritmo 7: Construção da Tabela SLR

Entrada: Autômato LR(0) M
Saida: Tabelas ACTION e GOTO

inicio

```

  ACTION  $\leftarrow$  Tabela(Estado, Terminal, Ação)
  GOTO  $\leftarrow$  Tabela(Estado, Não-terminal, Estado)
  para cada estado  $s_i \in M$  fazer
    para cada item  $[A \rightarrow \alpha \cdot a\beta] \in s_i$  fazer
       $s_j \leftarrow \delta(s_i, a)$ 
      se  $s_j \neq \emptyset$  então
        ACTION[ $s_i, a$ ]  $\leftarrow$  "shift  $s_j$ "
      fim
    fim
    para cada item  $[A \rightarrow \alpha \cdot] \in s_i$  fazer
      para cada  $a \in \text{FOLLOW}(A)$  fazer
        ACTION[ $s_i, a$ ]  $\leftarrow$  "reduce  $A \rightarrow \alpha$ "
      fim
    fim
    para cada não-terminal  $A$  fazer
       $s_j \leftarrow \delta(s_i, A)$ 
      se  $s_j \neq \emptyset$  então
        GOTO[ $s_i, A$ ]  $\leftarrow s_j$ 
      fim
    fim
  fim
  retornar ACTION, GOTO

```

fim

APÊNDICE H - ALGORITMO ANÁLISE SLR

Algoritmo 8: Análise SLR

Entrada: Tabelas ACTION, GOTO, String de entrada w

Saida: Sucesso ou erro

inicio

$pilha \leftarrow [s_0, \$]$

$token \leftarrow$ próximo token de w

repetir

$s \leftarrow \text{topo}(pilha)$

se ACTION[$s, token$] = “shift s_j ” **então**

empilhe(s_j)

$token \leftarrow$ próximo token de w

senão, se ACTION[$s, token$] = “reduce $A \rightarrow \beta$ ” **então**

desempilhe($2 \times |\beta|$), remove $|\beta|$ estados e $|\beta|$ símbolos

$s' \leftarrow \text{topo}(pilha)$

empilhe(GOTO[s', A])

senão, se ACTION[$s, token$] = “aceitar” **então**

retornar sucesso

senão

retornar erro

fim

até que para sempre

fim

APÊNDICE I - ALGORITMO CLOSURE LR(1)

Algoritmo 9: Closure LR(1)

Entrada: Gramática G , Estado s
Saida: Closure do estado s
inicio
 $added \leftarrow \text{verdadeiro}$
enquanto $added$ **fazer**
 $added \leftarrow \text{falso}$
para cada item $[A \rightarrow \alpha \cdot B\beta, \ell] \in s$ **fazer**
para cada produção $B \rightarrow \gamma \in G$ **fazer**
 $lookahead \leftarrow \text{FIRST}(\beta)$
se $\epsilon \in \text{FIRST}(\beta)$ **então**
 $lookahead \leftarrow lookahead \cup \{\ell\}$
fim
para cada $a \in lookahead$ **fazer**
se $[B \rightarrow \cdot\gamma, a] \notin s$ **então**
 $s \leftarrow s \cup \{[B \rightarrow \cdot\gamma, a]\}$
 $added \leftarrow \text{verdadeiro}$
fim
fim
fim
fim
retornar s
fim

APÊNDICE J - ALGORITMO CONSTRUÇÃO DO AUTÔMATO LR(1)

Algoritmo 10: Construção do Autômato LR(1)

Entrada: Gramática G
Saida: Autômato M
inicio
 $s_0 \leftarrow \{[S' \rightarrow \cdot S, \$]\}$
 $\text{CLOSURE}(s_0)$
 $M \leftarrow \{s_0\}$
 $\text{newStates} \leftarrow \{s_0\}$
para cada $s \in \text{newStates}$ **fazer**
para cada $X \in T \cup NT$ **fazer**
 $s' \leftarrow \emptyset$
para cada item $[A \rightarrow \alpha \cdot X\beta, \ell] \in s$ **fazer**
 $s' \leftarrow s' \cup \{[A \rightarrow \alpha X \cdot \beta, \ell]\}$
fim
se $s' \neq \emptyset$ **então**
 $\text{CLOSURE}(s')$
se $s' \notin M$ **então**
 $M \leftarrow M \cup \{s'\}$
 $\text{newStates} \leftarrow \text{newStates} \cup \{s'\}$
fim

 adicione transição $s \xrightarrow{X} s'$
fim
fim
 $\text{newStates} \leftarrow \text{newStates} - \{s\}$
fim
retornar M
fim

APÊNDICE K - ALGORITMO CONSTRUÇÃO DA TABELA LR(1)

Algoritmo 11: Construção da Tabela LR(1)

Entrada: Autômato LR(1) M
Saida: Tabelas ACTION, GOTO

inicio

```

para cada estado  $s_i \in M$  fazer
  para cada item  $[A \rightarrow \alpha \cdot a\beta, \ell] \in s_i$  fazer
     $s_j \leftarrow \delta(s_i, a)$ 
    se  $s_j \neq \emptyset$  então
      | ACTION $[s_i, a] \leftarrow$  “shift  $s_j$ ”
    fim
  fim
  para cada item  $[A \rightarrow \alpha \cdot, \ell] \in s_i$  fazer
    para cada  $a \in \ell$  fazer
      | ACTION $[s_i, a] \leftarrow$  “reduce  $A \rightarrow \alpha$ ”
    fim
  fim
  se item  $[S' \rightarrow S \cdot, \$] \in s_i$  então
    | ACTION $[s_i, \$] \leftarrow$  “aceitar”
  fim
  para cada não-terminal  $A$  fazer
     $s_j \leftarrow \delta(s_i, A)$ 
    se  $s_j \neq \emptyset$  então
      | GOTO $[s_i, A] \leftarrow s_j$ 
    fim
  fim
fim
retornar (ACTION, GOTO)

```

fim
